

2024/07/31 Quantum Innovation 2025

Toward Scalable Multicore Fault-Tolerant Quantum Computers using Quantum Multiplexing

University College London, Keio University
Shin Nishio



Outline

1. Background and overview of fault-tolerant quantum computation

2. Modular architecture

Multiplexed Quantum Communication with Surface and Hypergraph Product Codes

Shin Nishio, Nicholas Connolly, Nicolò Lo Piparo, William John Munro, Thomas Rowan Scruby, Kae Nemoto

Quantum 9, 1613



3. Improving the throughput

Online Job Scheduler for Fault-tolerant Quantum Multiprogramming

Shin Nishio, Ryo Wakizaka, Daisuke Sakuma, Yosuke Ueno, Yasunari Suzuki

arXiv: 2505.06741



- Prime factor (super-polynomial speedup)

Classical : $\exp\left(\left((64/9)^{1/3} + o(1)\right) (\log n)^{1/3} (\log \log n)^{2/3}\right)$

Shor[1]: $O(n^3)$

- Database searching (polynomial speedup)

Classical: $O(n)$

Grover[2]: $O(\sqrt{n})$

Discrete logarithms, quantum simulation, system of linear equation...

The bigger the problem, the greater the advantage over the classics.

[1] Peter Shor "Algorithms for quantum computation: discrete logarithms and factoring." *Proceedings 35th annual symposium on foundations of computer science*. Ieee, 1994.

[2] Lov K. Grover "Quantum mechanics helps in searching for a needle in a haystack." *Physical Review Letters*, 79(2):325-328, 1997.

High performance computing[3]

Fugaku(2020)

442 PFLOPS

clock rate for a processor 2.2 GHz

Quantum Computing (superconducting + optical interconnect)[4]

Single qubit gate 30 ns (33 MHz)

Two qubit gate 60 ns (16 MHz)

Measurement 240 ns (4 MHz)

Entanglement generation for remote nodes 1000 ns (1 MHz)

→ 10^3 speedup is required!

quantum advantage requires solving large-scale problems

[3] 安里彰 「コデザインによるスーパーコンピュータ富岳プロセッサの開発」 電子情報通信学会 基礎・境界ソサイエティ Fundamentals Review (2022)

[4] Shin Nishio and Ryo Wakizaka. "InQulR: Intermediate Representation for Interconnected Quantum Computers." arXiv preprint arXiv:2302.00267 (2023).

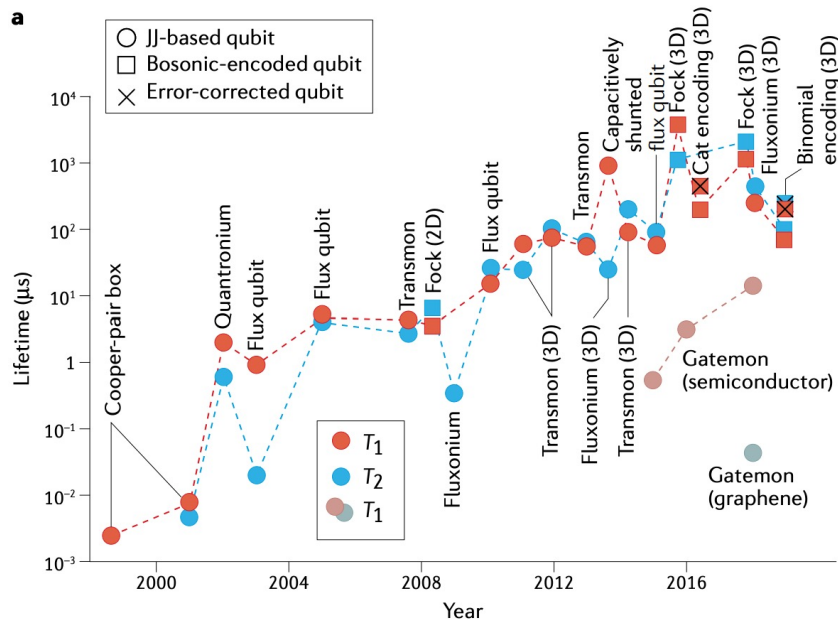
Lifetime and scaling

1

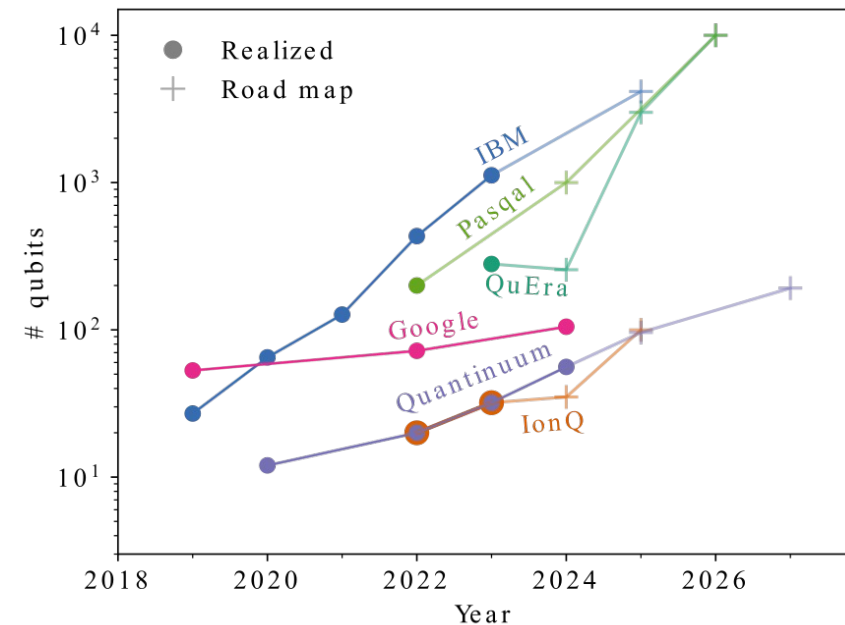
Qubit is still fragile, but # of qubits is scaling

→ Fault-tolerant quantum computing with Quantum Error Correction (QEC)

Lifetime of superconducting qubits [5]



of qubits in a processor [6]



[5] Irfan Siddiqi, "Engineering high-coherence superconducting qubits." *Nature Reviews Materials* 6.10 (2021): 875-891.

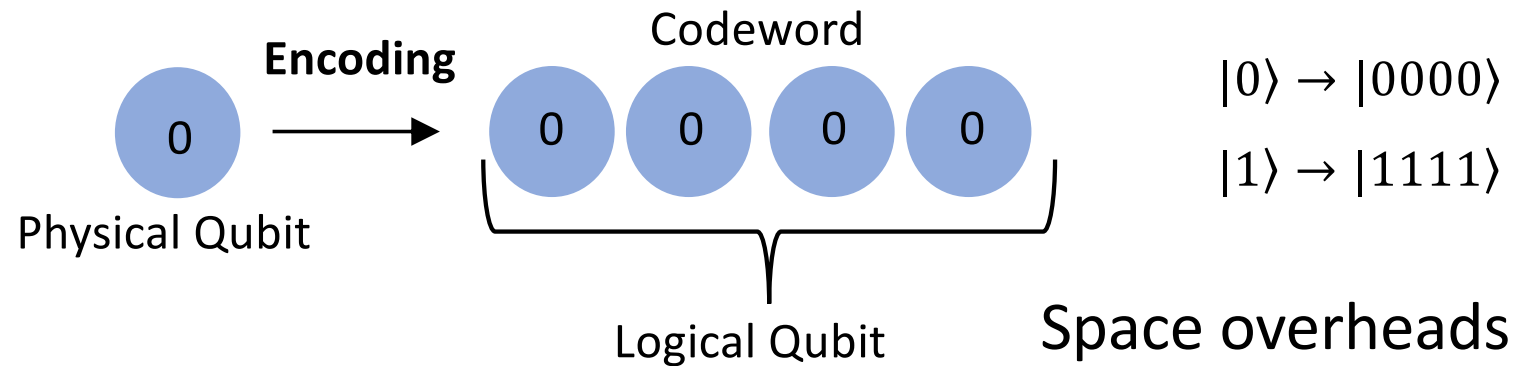
[6] Koen Groenland, "Introduction to Quantum Computing for Business" to be released in Q1 2025.

Quantum Error Correction Codes

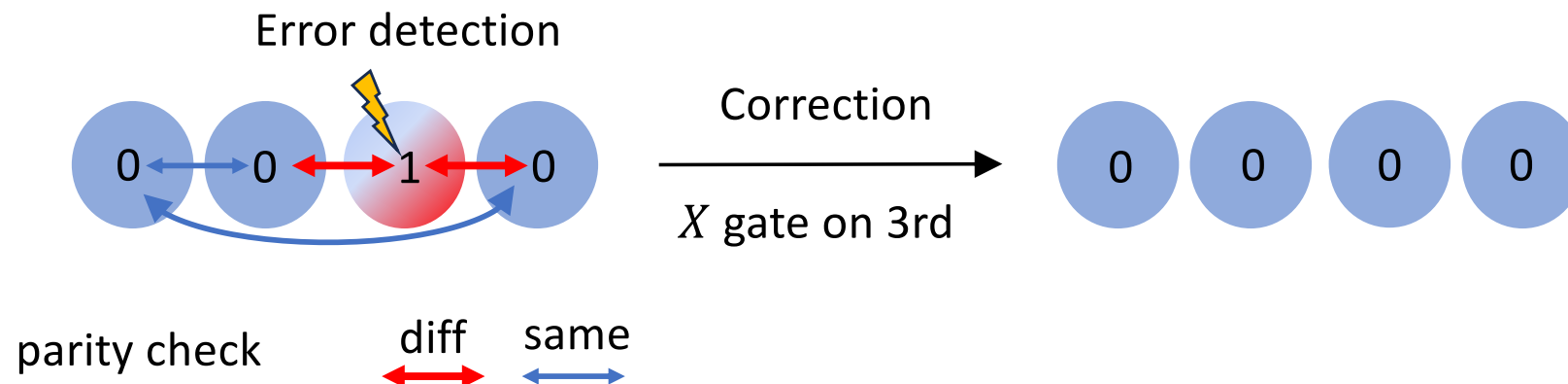
1

- Encode quantum information onto **redundant** system with Quantum Error Correction codes

e.g. $[[4,1,4]]$ Repetition code



- detect & correct errors



Estimated FTQC Overheads

1

Required size for quantum advantage

Application: a 2048 bit integer factoring with Shor's algorithm

Number of Qubits

[[1352,1,25]]

**10^6 physical qubits
(1600 logical qubits)**

Execution time

codecycle: $10^{-6}s$

5 days ($4.3 \times 10^5 s$)

Overheads for FTQC is the most significant barrier to the realization of FTQC

[7] Craig Gidney. "How to factor 2048 bit RSA integers with less than a million noisy qubits." *arXiv preprint arXiv:2505.15917* (2025).

- Reducing the required resources
 - Coding rate, logical gate, algorithms...

Today's talk

1. Exploring the feasibility of implementing on scalable architectures
 - Quantum Multiplexing for Modular architecture
2. Enhancing Throughput of the system
 - Quantum Multiprogramming

Limitation of single processor [8]

2

- Size of diffusion refrigerator
- Spaces for Wires

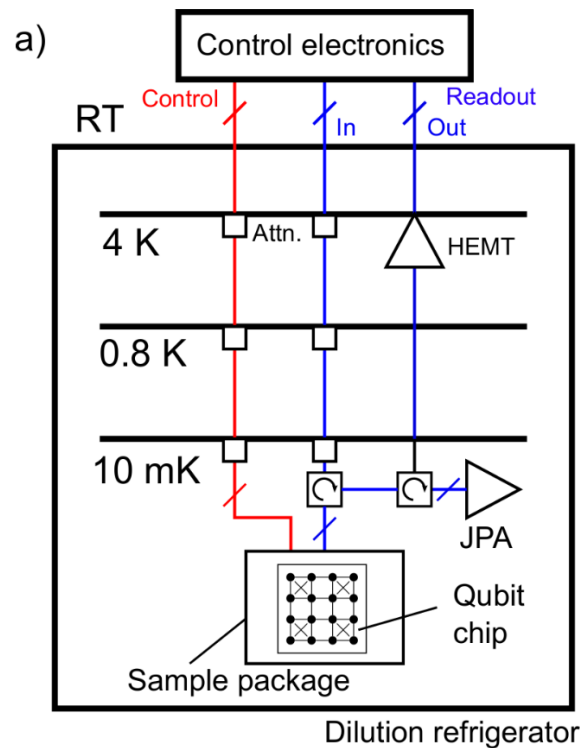
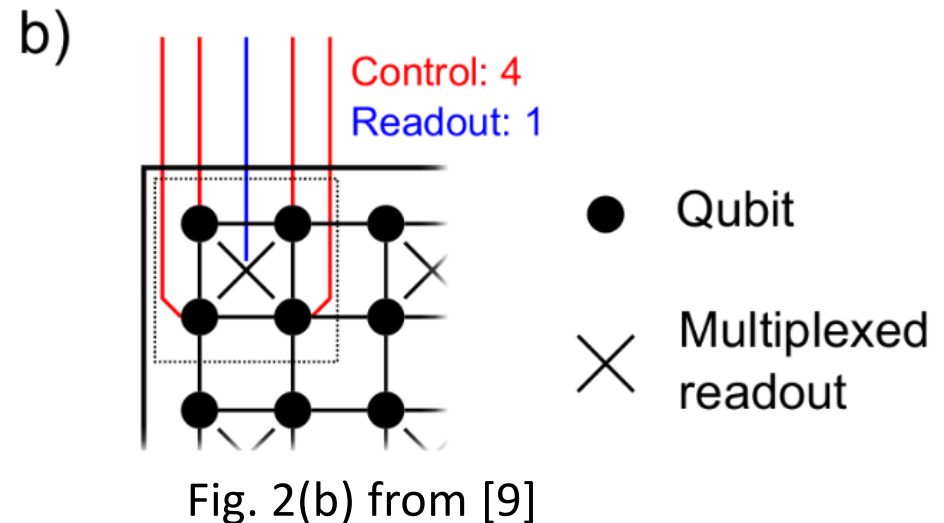


Fig. 2(a) from [8]



- Crosstalk errors
- Laser Addressing (trapped ion system)
- Heat
- Low throughput yield for dense chip

[8] Tamate, Shuhei, Yutaka Tabuchi, and Yasunobu Nakamura. "Toward realization of scalable packaging and wiring for large-scale superconducting quantum computers." *IEICE Transactions on Electronics* 105.6 (2022): 290-295.

Connecting Quantum Processors via Quantum Interconnects

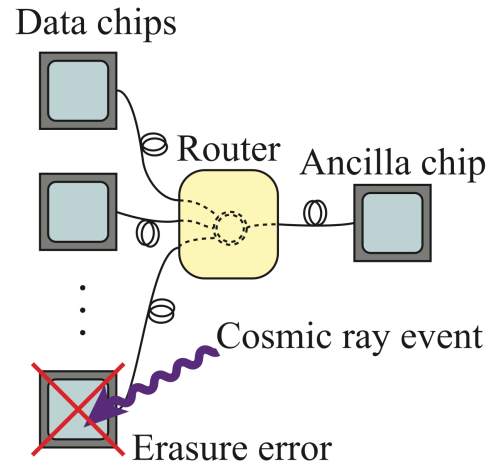


Fig. 1 in [9]

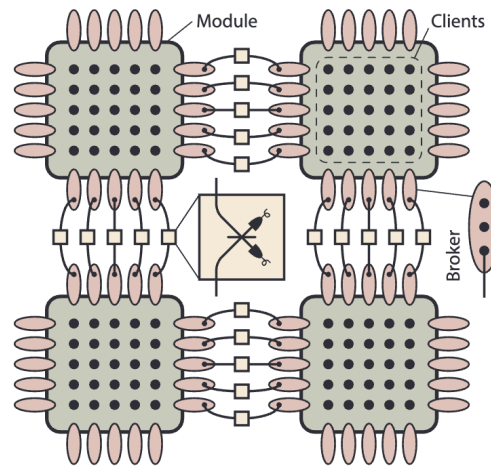


Fig. 1 in [10]

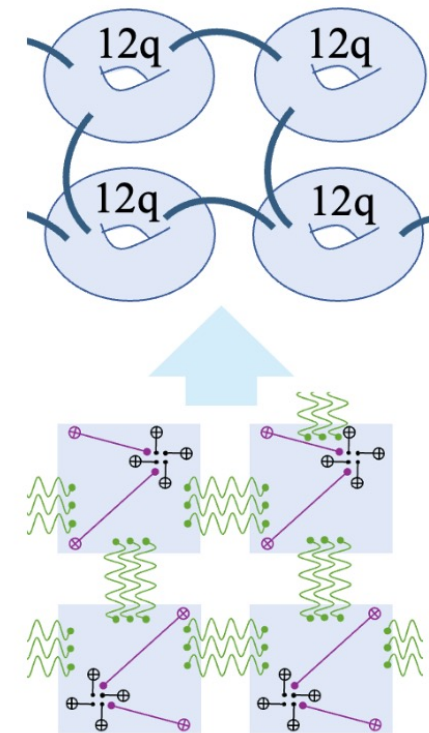


Fig. 1 in [11]



Scalable



Introduce a bottleneck of the system

[9] Qian Xu et al. "Distributed quantum error correction for chip-level catastrophic errors." *Physical review letters* 129.24 (2022): 240502.

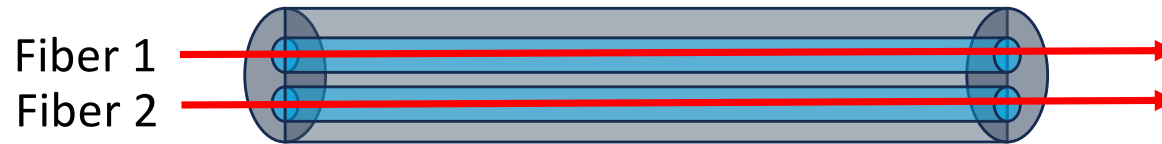
[10] Ying Li and Simon C. Benjamin. "Hierarchical surface code for network quantum computing with modules of arbitrary size." *Physical Review A* 94.4 (2016): 042303.

[11] Theodore J. Yoder et al. "Tour de gross: A modular quantum computer based on bivariate bicycle codes." *arXiv preprint arXiv:2506.03094* (2025).

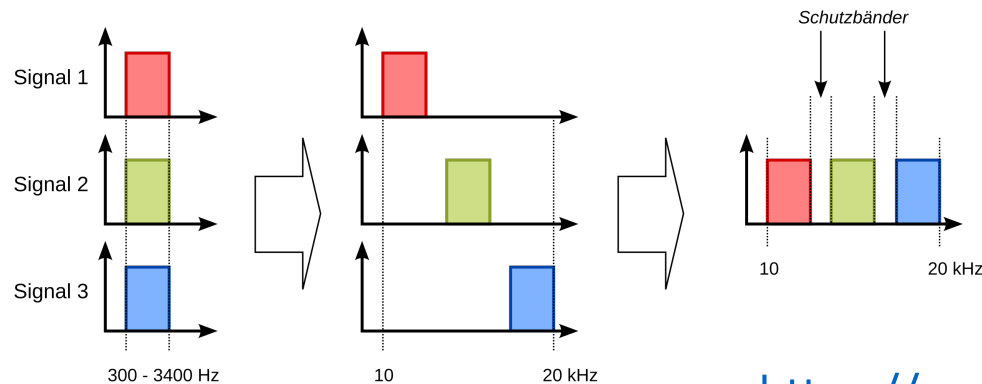
(Classical) Multiplexing

2

- Space Multiplexing

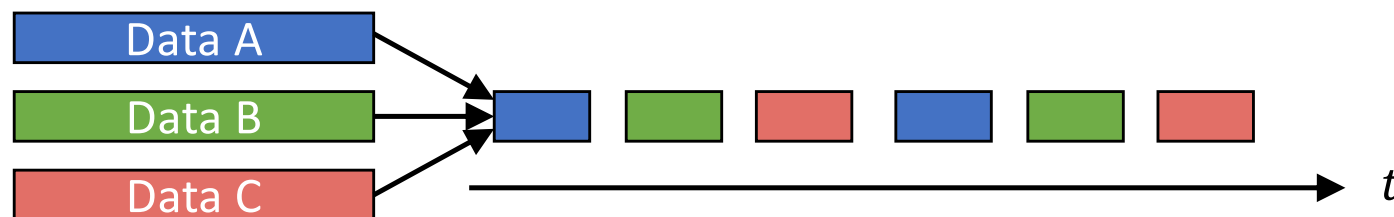


- Frequency Multiplexing



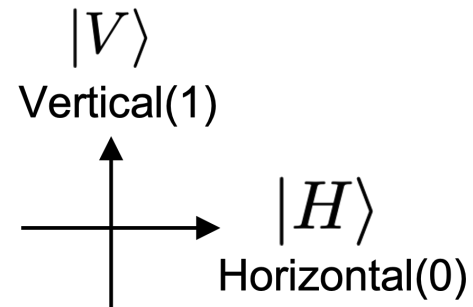
<https://en.wikipedia.org/wiki/Multiplexing>

- Time Multiplexing

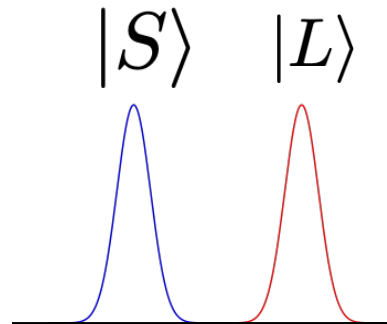


Treating the degrees of freedom of photons as qubits/qudits

- Polarization



- Timebin



timing of the photonic pulse

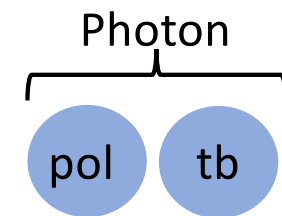
Quantum Multiplexing [12]: Encoding high dimensional information on to a single photon

e.g. polarization and timebin

$|VL\rangle, |VS\rangle, |HL\rangle, |VS\rangle$: encode 2 qubit

e.g. multiple components of timebin

$|LLL\rangle, |LLS\rangle, |LSL\rangle \dots |SSS\rangle$: encode 3 qubit



[12] Nicolò Lo Piparo, William J. Munro, and Kae Nemoto.

"Quantum multiplexing." *Physical Review A* 99.2 (2019): 022337.

Quantum Multiplexing

2

An experiment with $m = 5$

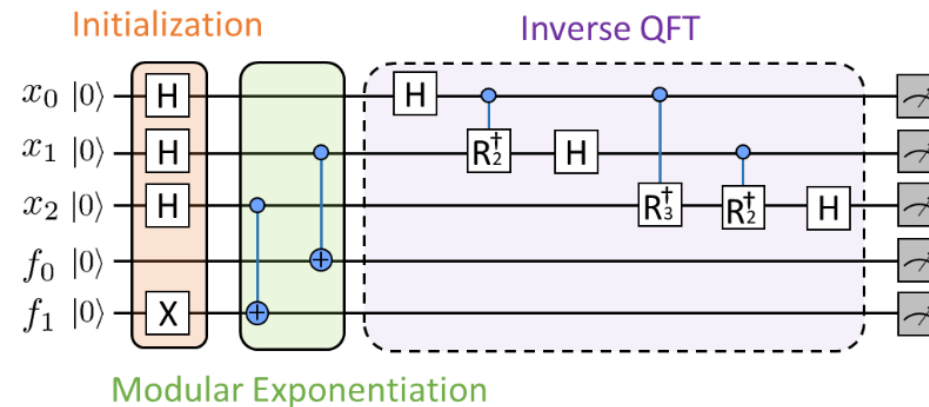
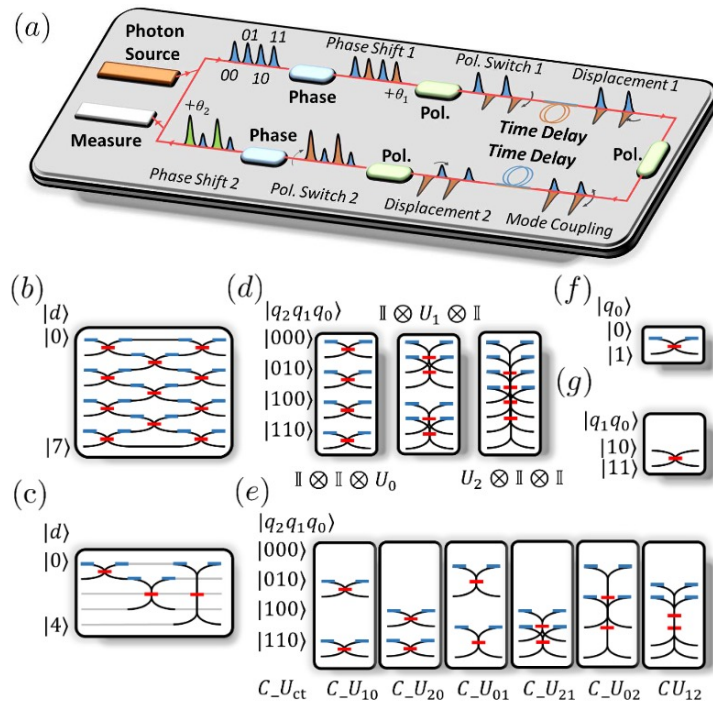


Fig from [13]

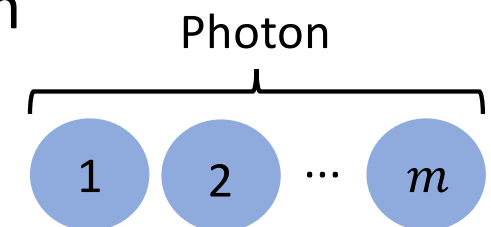
[13] Hao-Cheng Weng, Chih-Sung Chuu,
Implementation of Shor's Algorithm with a Single Photon in 32 Dimensions
arXiv:2408.08138

Resource reduction with QM

- Single photon sources [14]
- # of CX gates for Toffoli gates [15]
- Gates for an encoder for Quantum Reed-Solomon Codes[16]
 - # of CX gates for Multi-Controlled gates [16]

We apply quantum multiplexing to surface code communication for easing the bottleneck in quantum interconnect

- assume to encode m qubits into a single photon



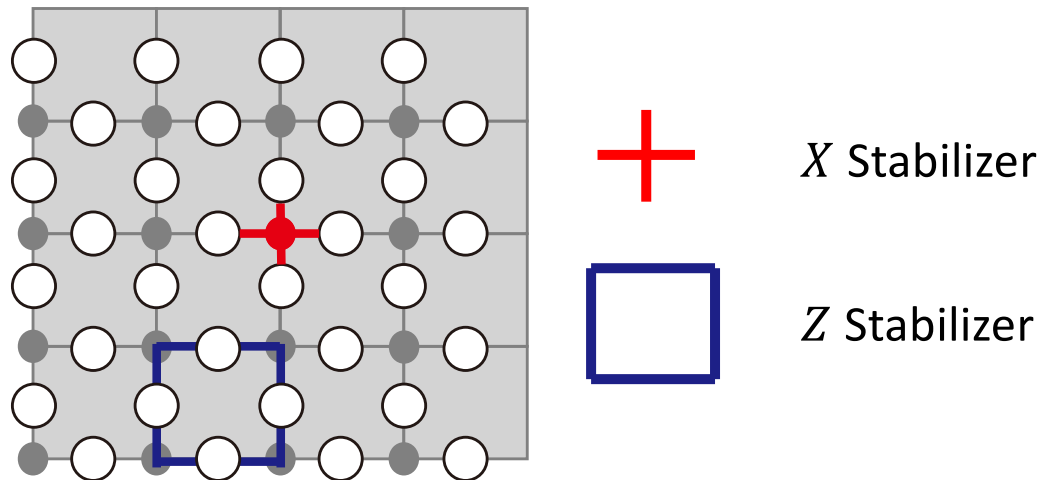
[14] Nicolò Lo Piparo, et al. "Resource reduction for distributed quantum information processing using quantum multiplexed photons." *Physical Review Letters* 124.21 (2020): 210503.

[15] Nicolò Lo Piparo, et al. "Aggregating quantum networks." *Physical Review A* 102.5 (2020): 052613.

[16] Shin Nishio et al. "Resource reduction in multiplexed high-dimensional quantum Reed-Solomon codes." *Physical Review A* 107.3 (2023): 032620.

A quantum error correction code

- 2D implementable, high threshold



- Erasure(photon loss) is dominant error source in the optical systems

$$\rho \rightarrow (1 - \varepsilon)\rho + \varepsilon|e\rangle\langle e|$$

Erasure error

where $|e\rangle \notin \mathcal{H}_2$

→ We can detect erasure without destruction!

Erasure error correction

0. detect erasure errors

1. replace the erased qubit with a mixed-state

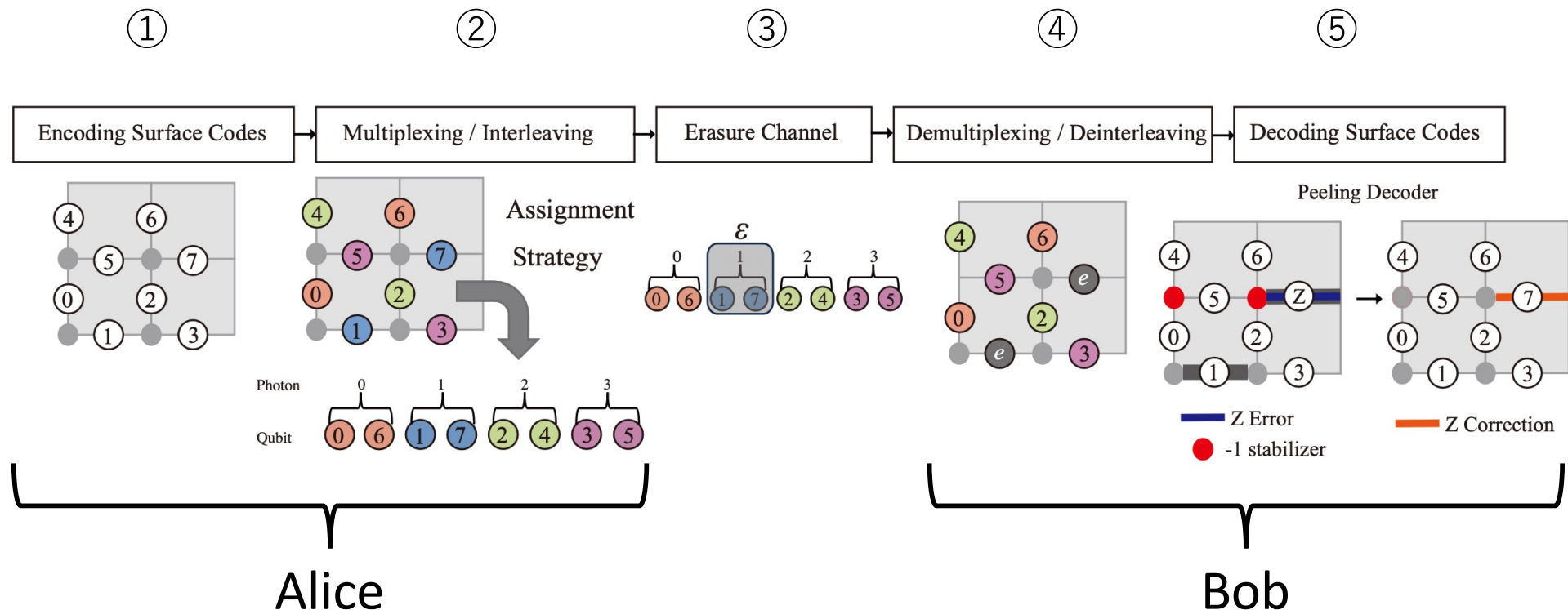
$$\frac{\mathbb{I}}{2} = \frac{1}{4}(\rho + X\rho X + Y\rho Y + Z\rho Z)$$

2. apply stabilizer measurement

3. decoding (calculate syndrome and correct)

We propose multiplexed SC comm

- Sending multiplexed codewords
- sometimes it cause a correlated error inside a codeword

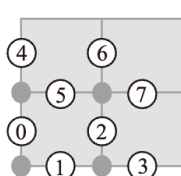
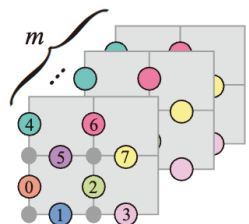
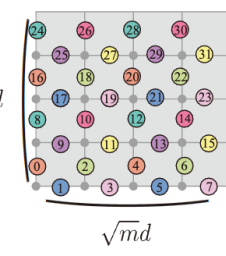
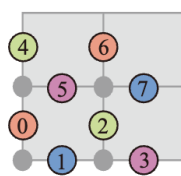


Three scenarios for QM Comm

2

multiplexing with m (# of qubits in a photon)

- (A) m different code words with the same # of photons
- (B) $\sqrt{m} \times \sqrt{m}$ bigger code words with the same # of photons
- (C) Original code words with $\frac{1}{m}$ photons

Scenarios	 without multiplexing	 (A)	 (B)	 (C)
Code parameters	$[[2d^2, 1, d]]$	$[[2d^2, 1, d]]$	$[[2md^2, 1, \sqrt{md}]]$	$[[2d^2, 1, d]]$
Number of Codes	1	m	1	1
Number of Data Qubits	$2d^2$	$2md^2$	$2md^2$	$2d^2$
Number of Photons	$2d^2$	$2d^2$	$2d^2$	$\lceil 2d^2/m \rceil$
Logical Error Rate	-	Same as without quantum multiplexing	Affected by classical correlation	Affected by classical correlation

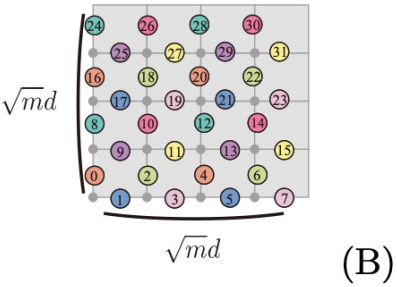
Scenario (B) larger codeword

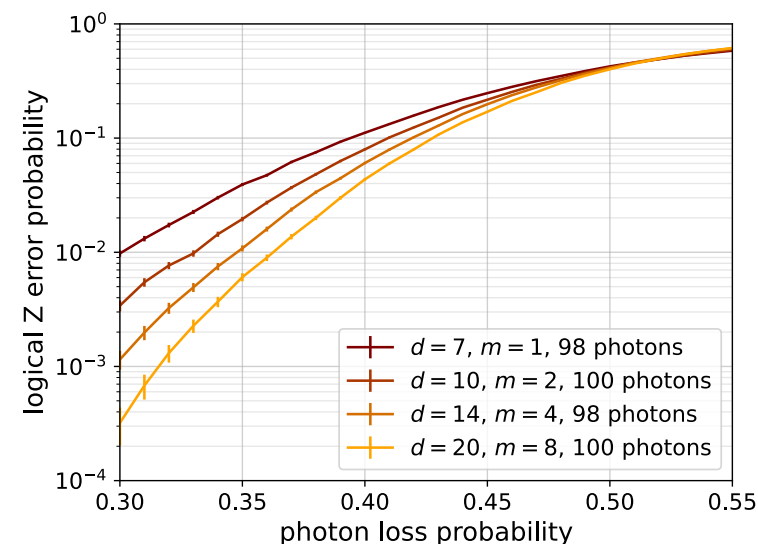
2

Sending bigger codes

- Large code distance
 - Encoding m qubits in a photon
- correlated Pauli errors

Benefit from the large distance is dominant

Scenarios	
Code parameters	$[[2md^2, 1, \sqrt{md}]]$
Number of Codes	1
Number of Data Qubits	$2md^2$
Number of Photons	$2d^2$
Logical Error Rate	Affected by classical correlation

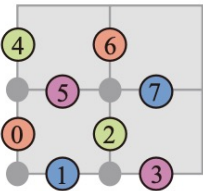


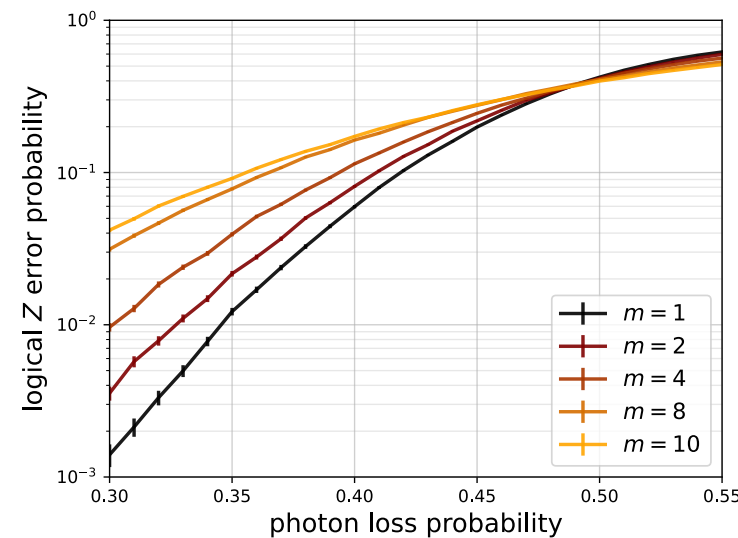
Scenario (C)

2

Sending same codes with fewer photons

- m times better throughput
 - Encoding m qubits in a photon
- Classical correlation inside codewords

Scenarios	 (C)
Code parameters	$[[2d^2, 1, d]]$
Number of Codes	1
Number of Data Qubits	$2d^2$
Number of Photons	$\lfloor 2d^2/m \rfloor$
Logical Error Rate	Affected by classical correlation

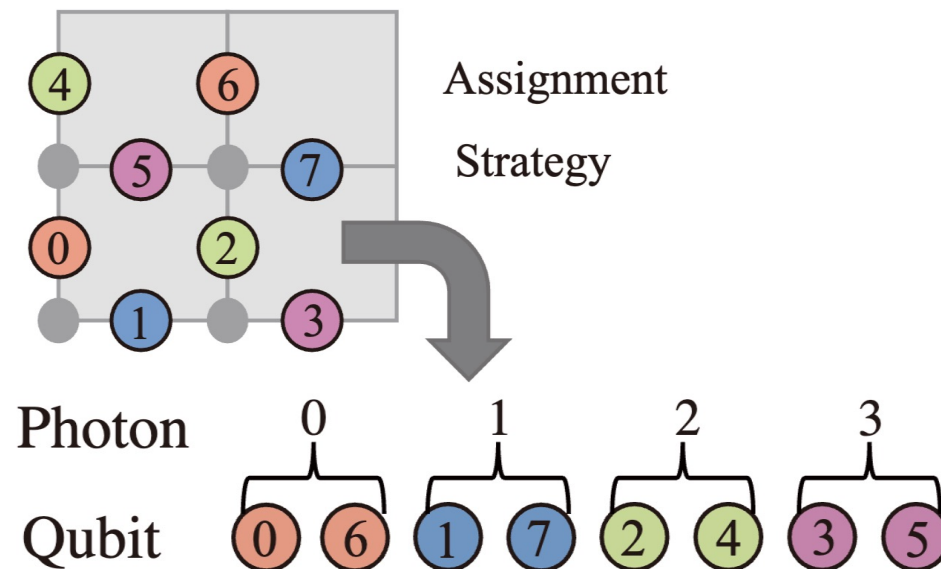


- There is a degree of freedom in the assignment of qubits to photons

→ We propose 5 assignment strategies

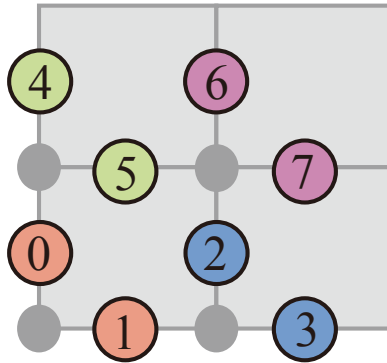
The impact of introducing correlation on the performance is not trivial.

- Can negative effects be avoided with assignment?

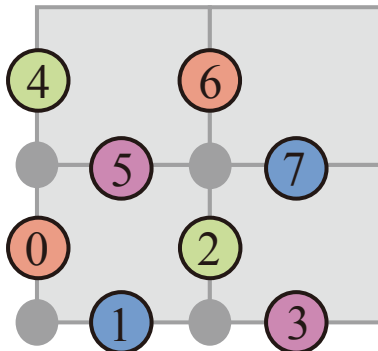


Assignment strategies for QM

2

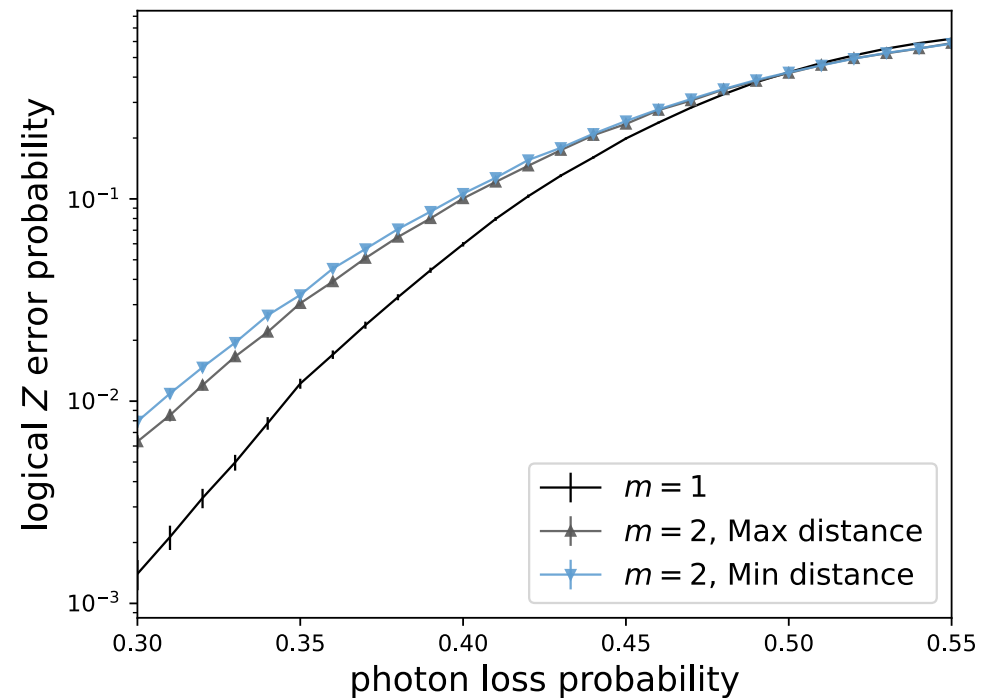


(i) Minimum distance



(ii) Maximum distance

MC simulation result



Maximizing distance may help!

(iii) Uniformly random

Inspired by interleaving for classical error correction codes

(iv) Random + Thresholds

Hiring randomness and trying to maximize the distance between qubits in a single photon

For photons:

Set threshold

$$T = \frac{d}{2} - 1$$

Pick 1st qubit randomly

While # of qubits in the photon $< m$:

Pick a candidate qubit c randomly.

if c has distance $> T$ from nearest member:

add c to the photon

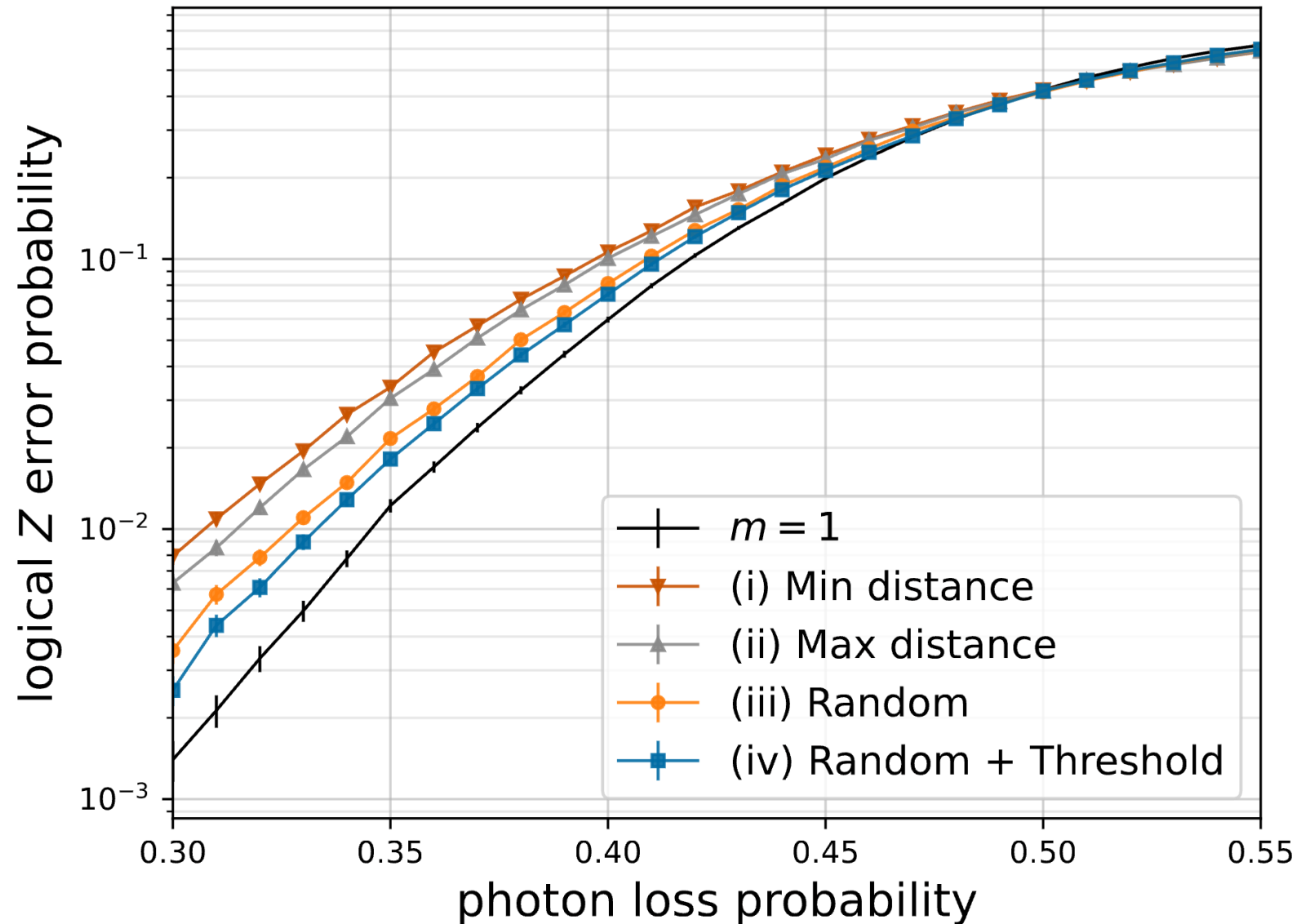
if there is no candidate:

update T as $T - 1$

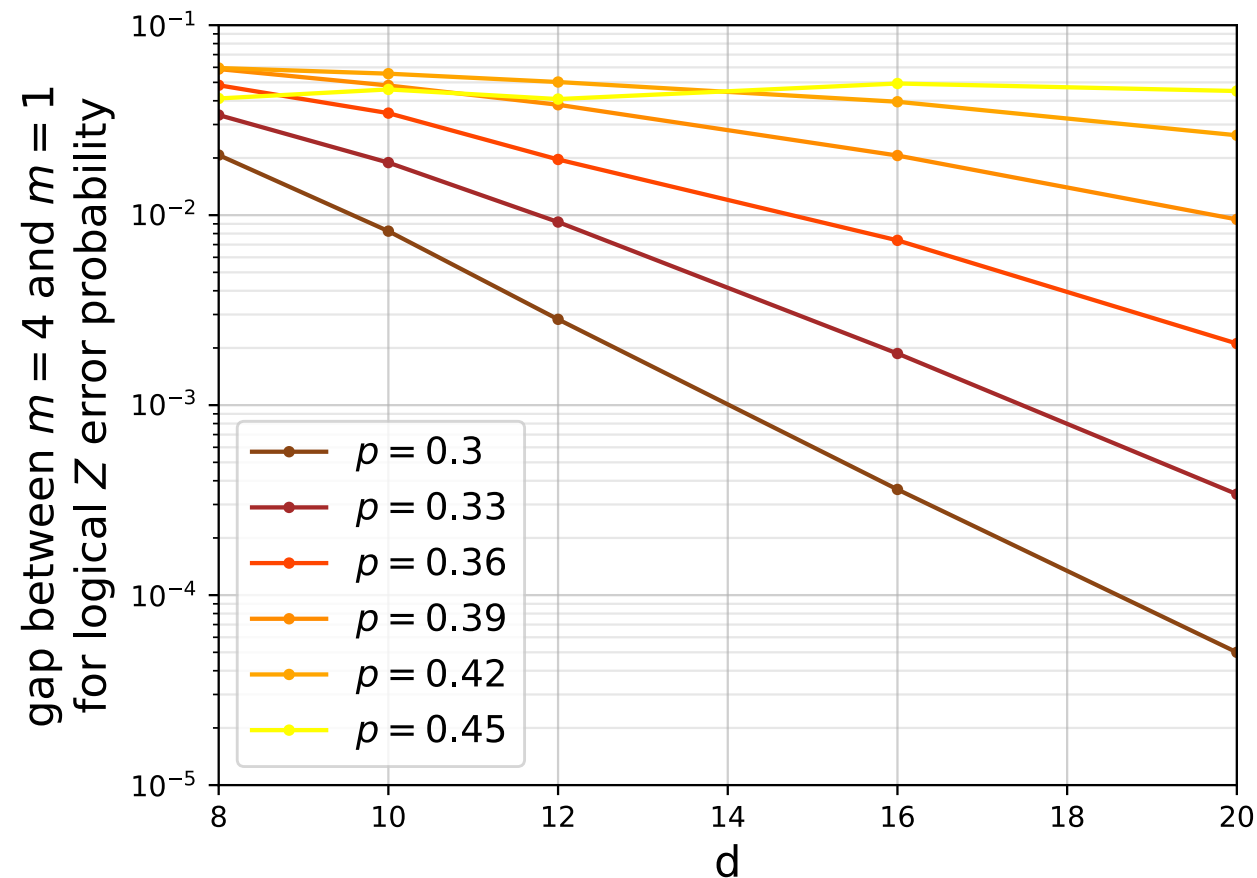
Assignment strategies for QM (2)

2

[[200, 2, 10]]



If $m \ll d$, classical correlation can be ignored



- 👍 QM reduces the required number of resources
- Number of gates → Fast encoding
 - Number of photons → Increase the throughput

QM may introduce a correlation between errors in qubits

- could be dealt with assignment strategies or higher distance
-
- Other assignment strategy
 - Efficiency of conversion from qubits to photons
 - Other multiplexing methods



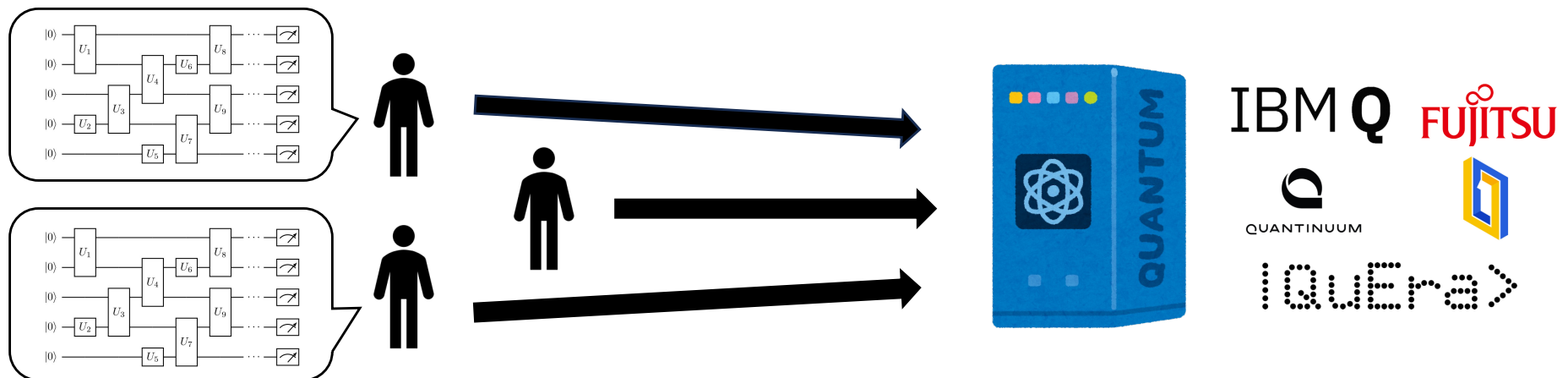
Parallel execution of multiple programs

👍 Increased throughput → economical benefit

👎 Resource contention (noise, competing for magic states, etc.)

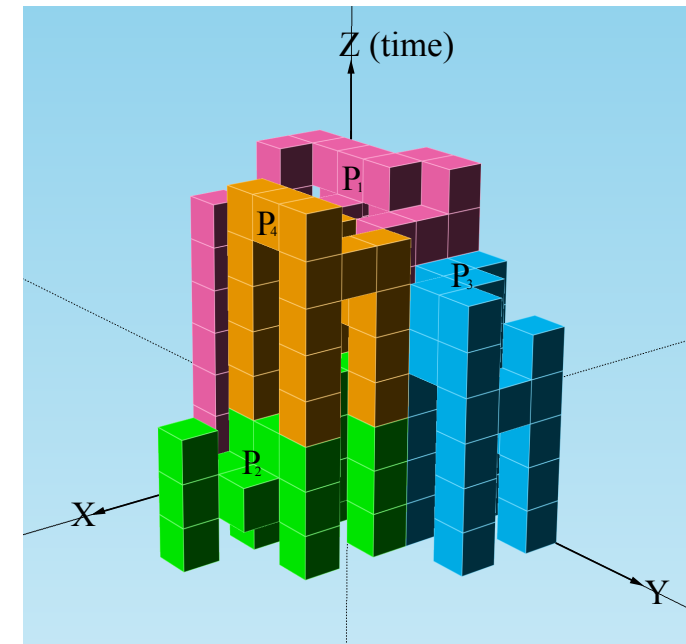
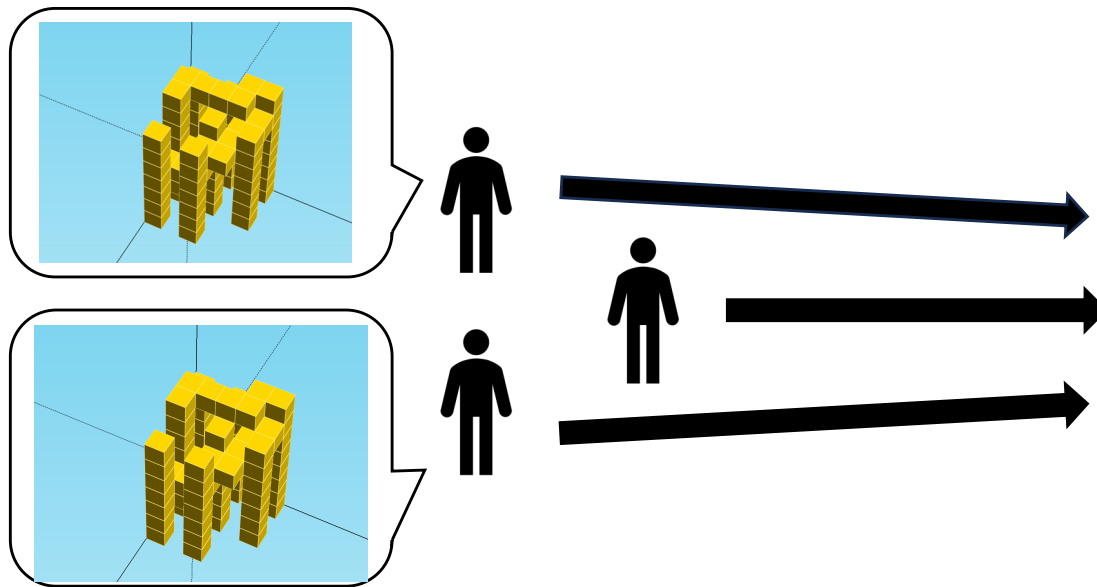
👎 Need a scheduler and allocator of shared resources

- Computational resources of scheduler



Here we assume

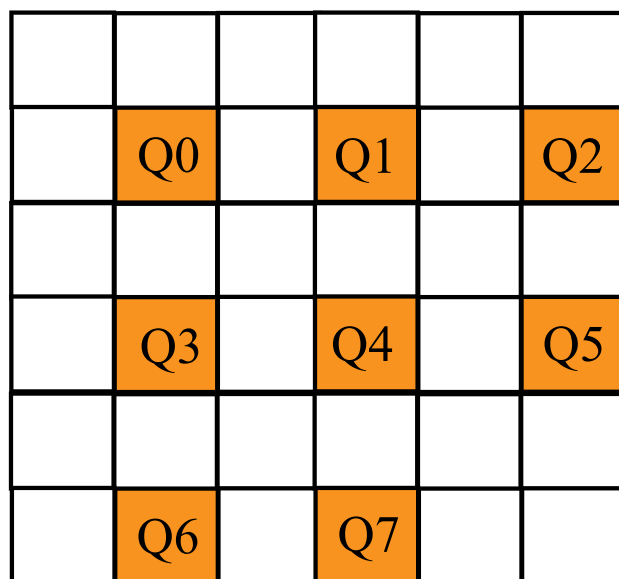
- QMP for a FTQC system
- Jobs are given as 3D polycubes:
lattice surgery circuit on surface codes



Quantum Circuit with 8 qubits

```
CX[Q0, Q2];  
CX[Q1, Q6];  
CX[Q5, Q7];  
...
```

Floorplan

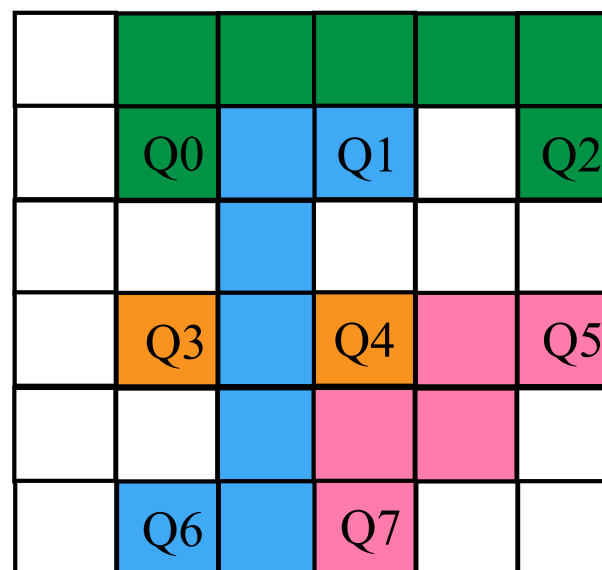


Compile



Connecting qubits → CX gate

A time slice of a LS circuit



Lattice Surgery

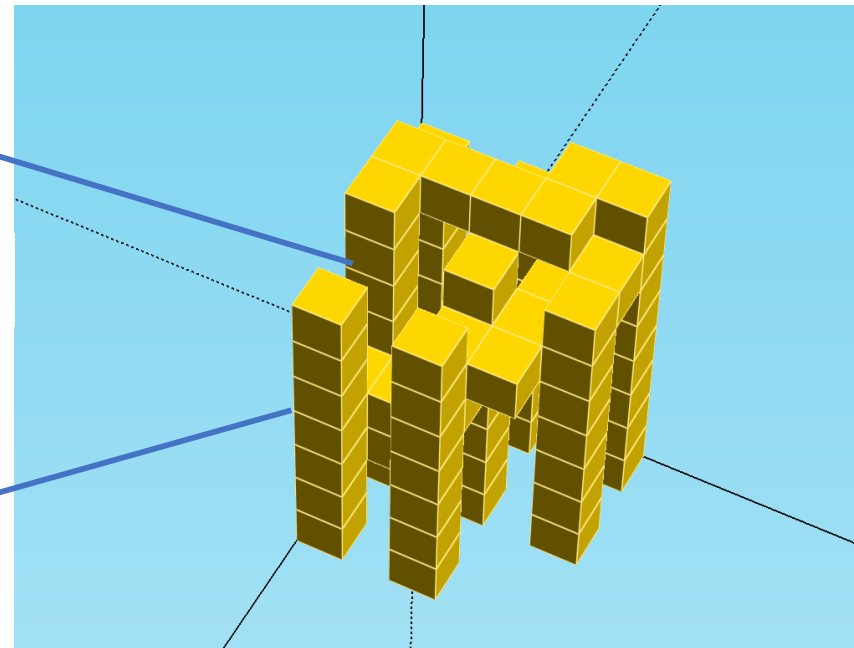


- LSs need not to collide
- parallel LS → faster execution

A time slice

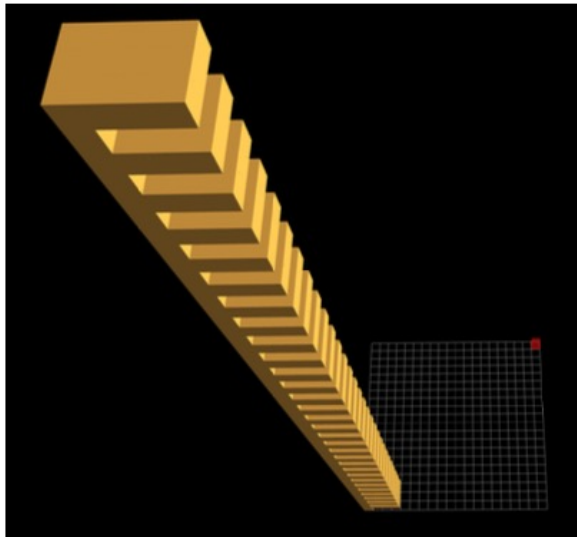
	Q0		Q1		Q2
	Q3		Q4		Q5
	Q6		Q7		

Lattice Surgery

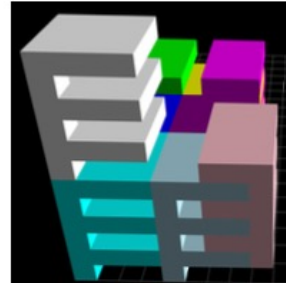


Stacking time slices → Poly cubes

Polycube  × 10

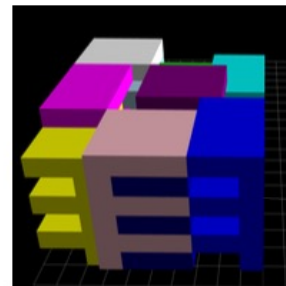


Greedy
→



10-60x (processor size dependent) compared to no multiprogramming

ILP
→



Against Greedy
1.7 times better throughput

- Offline Scheduling

→ You can't add new jobs during the computation

[20] 第9回量子ソフトウェア研究発表会

「マルチプログラミングによるFTQCのスループット向上」

中山 昌勲 (大阪大学), 鈴木 泰成 (NTT), 徳永 裕己 (NTT)

- Poly cubes fall
- 3D bin packing
- Past placements cannot be changed.



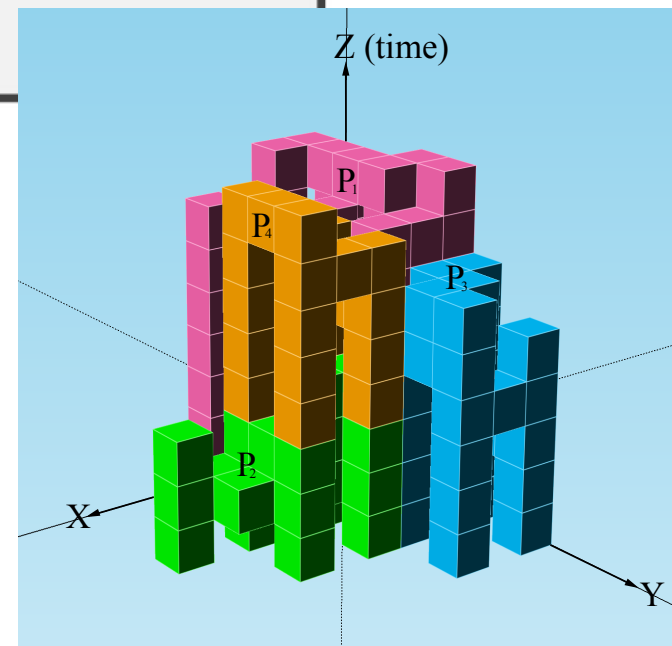
GEOM CUBE(PS1)

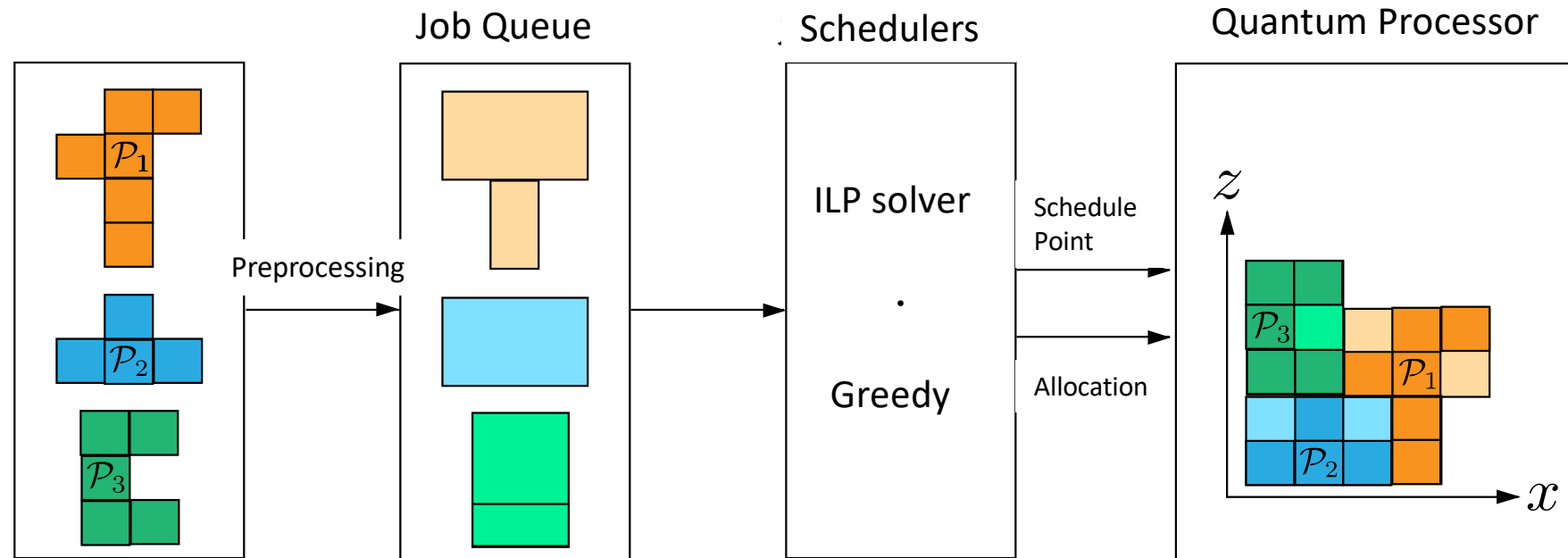


Problem 1 (Online scheduling): Given an input sequence $\mathcal{P}_1, \dots, \mathcal{P}_n$, where $\mathcal{P}_i$ is the i -th request represented by a polycube. The aim is to find the schedule $S_1, \dots, S_n$ that minimizes the total execution time $\max_i \{t_{\text{fin}}(S_i(\mathcal{P}_i))\}$ where each schedule S_i contains the allocation information, which consists of the 3D positions, rotation and whether the program will be flipped.

Polycube $\mathcal{P}_1, \dots, \mathcal{P}_n$

Schedules $S_1, \dots, S_n$





- Preprocessing: Convert programs into sufficient conditions

- Schedule Point

Minimum time at which the program can be placed in the next scheduling
Current time + average response time taken so far

If the scheduler does not respond by the scheduled point
→ Stall program execution and wait

- Integer Linear Programming (ILP) Problem Solver

Problem 2 (ILP problem based on cuboids):

$$\text{minimize } v \quad (1)$$

$$\text{subject to} \quad (2)$$

$$\forall i, j \in \{1, \dots, n\}.$$

$$a_{ij} + a_{ji} + b_{ij} + b_{ji} + c_{ij} + c_{ji} \geq 1 \quad (3)$$

$$X_i + W(a_{ij} - 1) \leq x_j - x_i \quad (4)$$

$$Y_i + H(b_{ij} - 1) \leq y_j - y_i \quad (5)$$

$$Z_i + T(c_{ij} - 1) \leq z_j - z_i \quad (6)$$

$$\forall i \in \{1 \dots, n\}.$$

$$x_i + X_i \leq W \quad (7)$$

$$y_i + Y_i \leq H \quad (8)$$

$$z_i + Z_i \leq T \quad (9)$$

$$z_i + Z_i \leq v \quad (10)$$

W/H : Processor width/height

X_i, Y_i, Z_i : size of $\mathcal{P}_i$

x_i, y_i, z_i : $\mathcal{P}_i$ Coordinate
(non-negative int)

End time of the program group :

$$T := \sum_i Z_i$$

Decision Variable

Collision between $\mathcal{P}_i, \mathcal{P}_j$

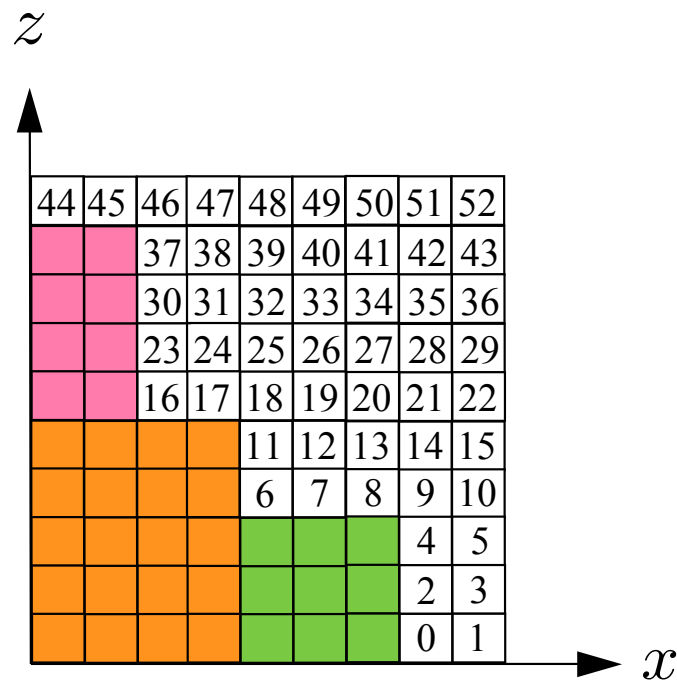
$$a_{ij}, b_{ij}, c_{ij}$$

- **Naïve Greedy**

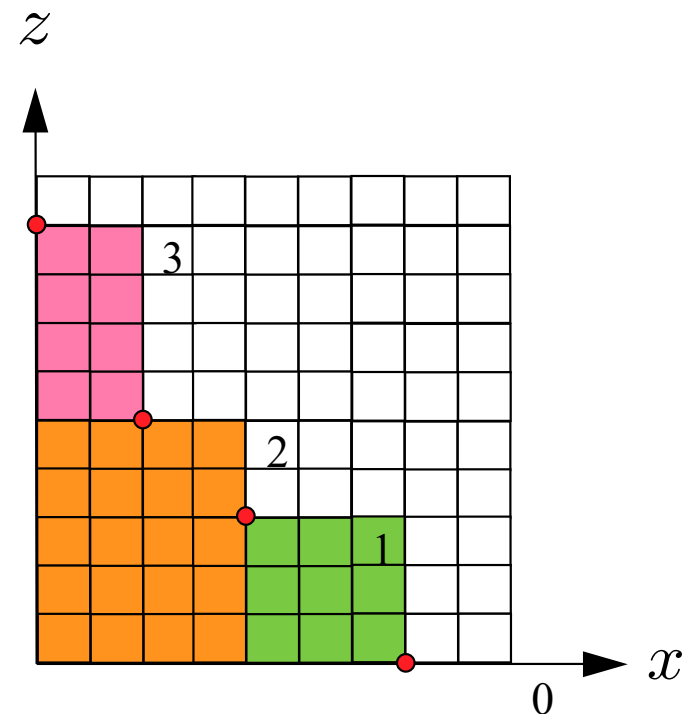
(z, x, y) in lexicographic order and the first place where it can be placed with minimum height.

- **Corner Greedy**

explore from the corner and place it first.



Naïve Greedy



Corner Greedy

Simulation with Rust

20×20 Medium-size QASM Bench

	ILP	Naive Greedy	Corner Greedy
Responsiveness(μ s)	224520	84141500	980

Type	B	Mean	Min	Max	StdDev	
ILP-C	5	244918	4962	14000017	831126	
ILP-C	10	1646150	70221	21186320	1823412	
ILP-C	15	5778808	553281	30102046	6098884	
ILP-C	20	24384941	3230509	30086498	9427292	
CG	5	2652	13	15989	2652	
CG	10	5529	24	17444	4990	
CG	15	7948	50	25567	7190	
CG	20	10900	88	31623	9789	(μ s)

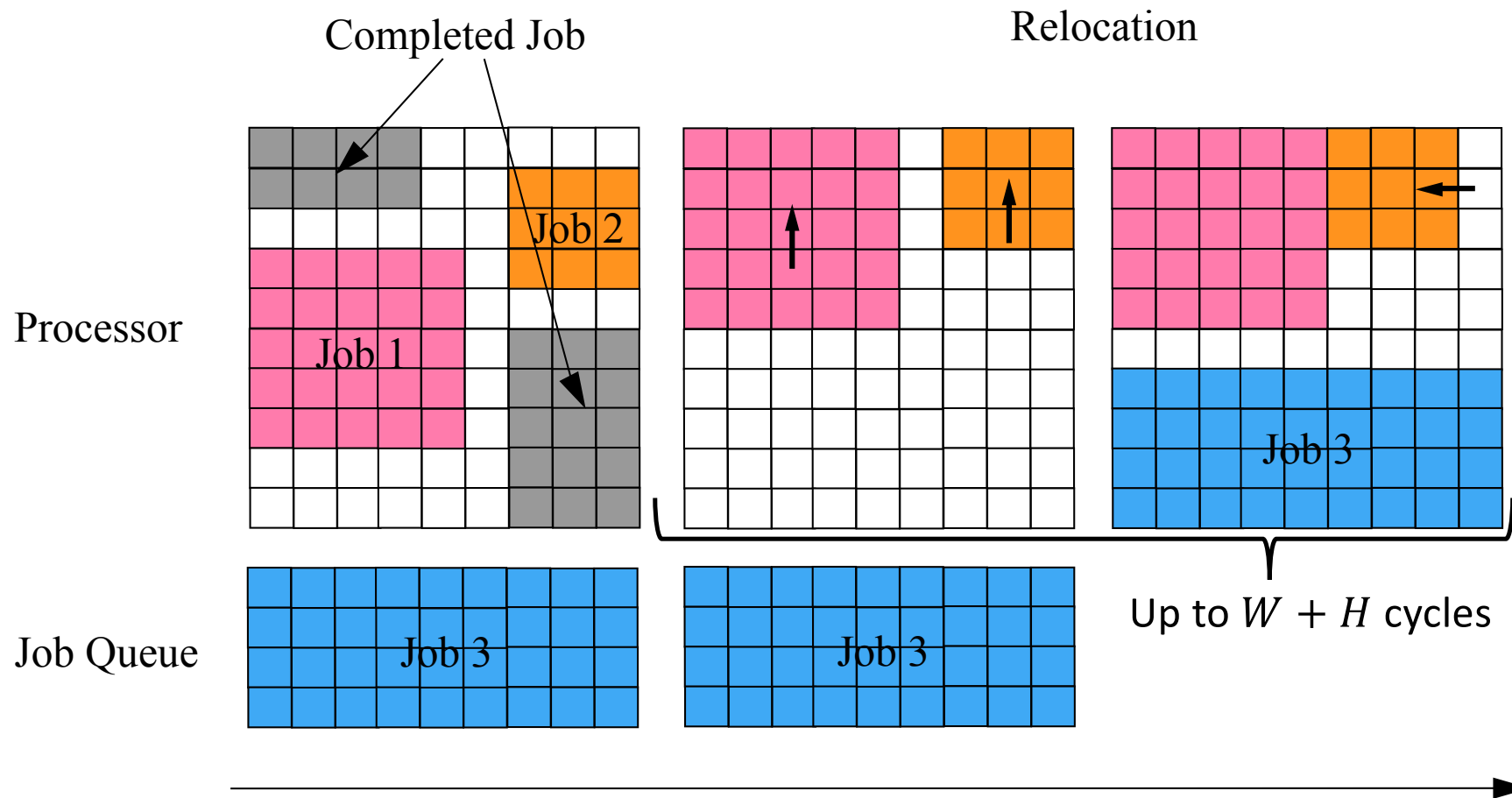
Corner Greedy is faster and easy to predict computation time

Defragmentation

3

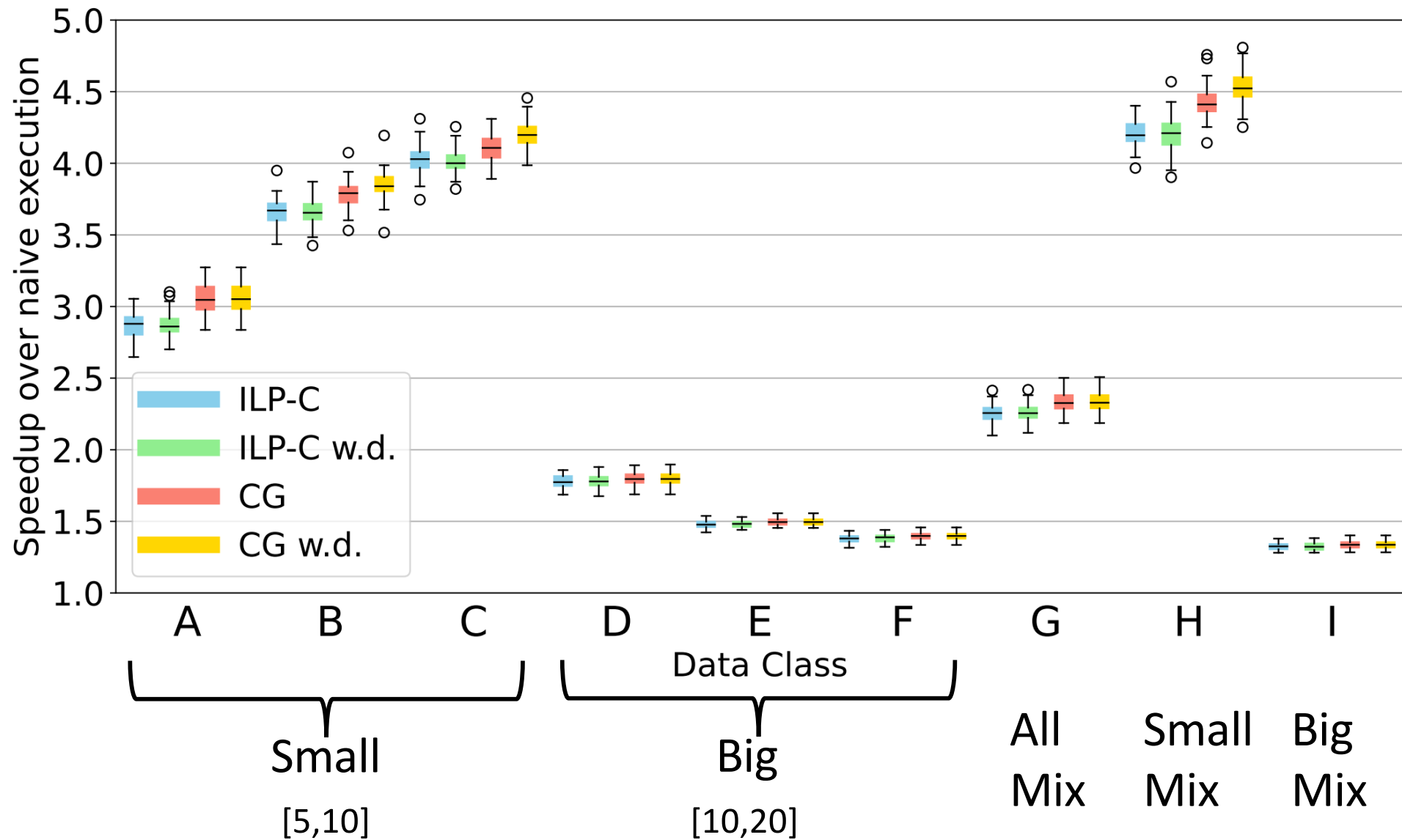
Relocate jobs during execution → reduce unnecessary gaps

- Execute when multiple jobs are close to the end timing



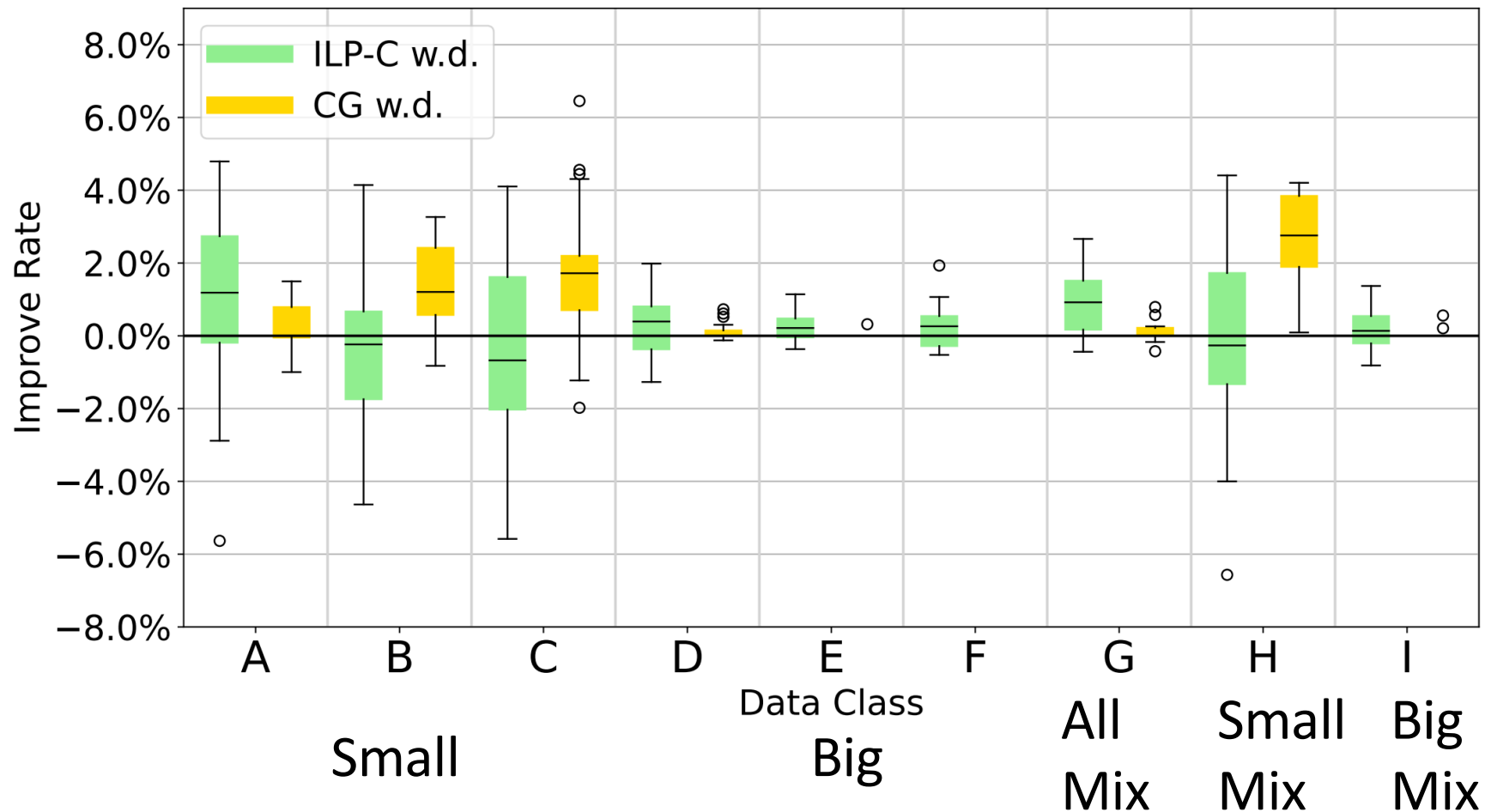
Throughput

3



Chip size: 20×20 patches

40



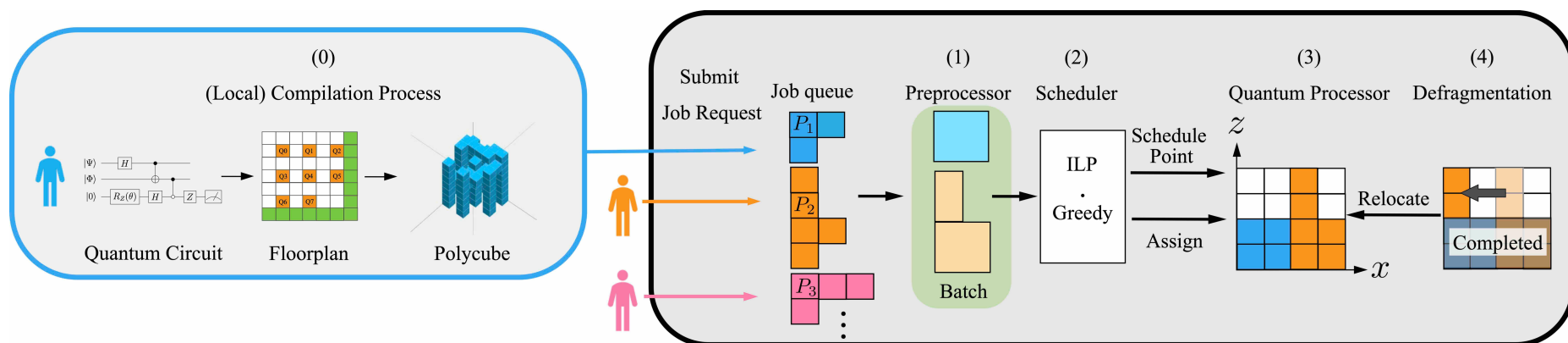
ILP-C does not necessarily improve performance
CG basically improves performance

To achieve quantum multiprogramming with high availability,

- Formulate an online scheduling problem
- Implemented ILP and heuristic schedulers
- Evaluate Responsiveness and Throughput
- Defragmentation

Benchmark circuits, toy compiler, visualization (open scad), scheduler, etc.

We can make an online scheduler with enough responsiveness



Conclusion

- Online scheduling ✓
- Management and movement of shared resources
 - fairness
- Decoder assignment to each logical qubit
- What should be handled statically/ dynamically?

Using other codes: scalable as long as it is a 2D chip implementation



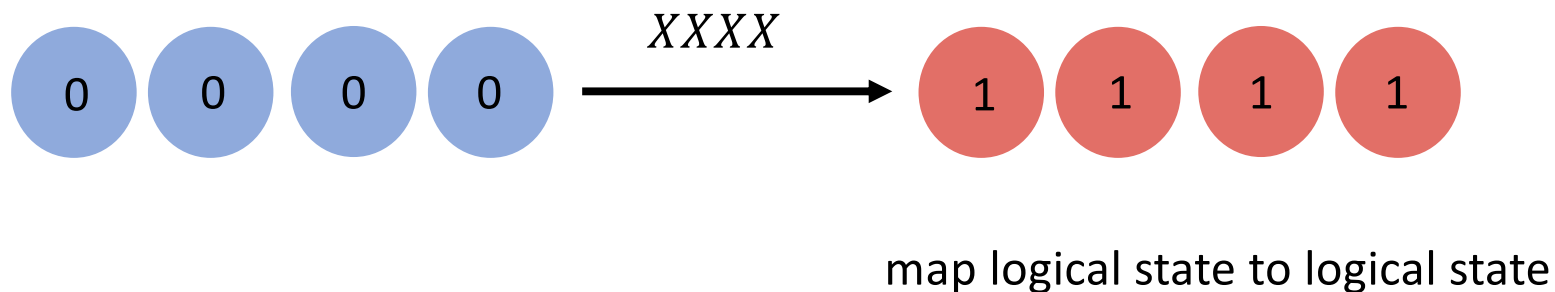
Appendices

- Perform computation on logical state of a quantum error correction codes

Physical gate

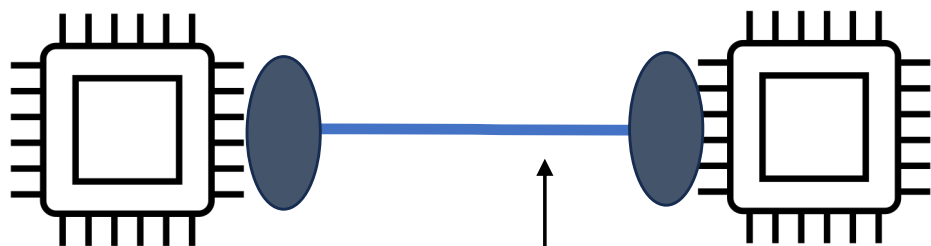


Logical gate



- Avoid error propagation

Quantum Processor



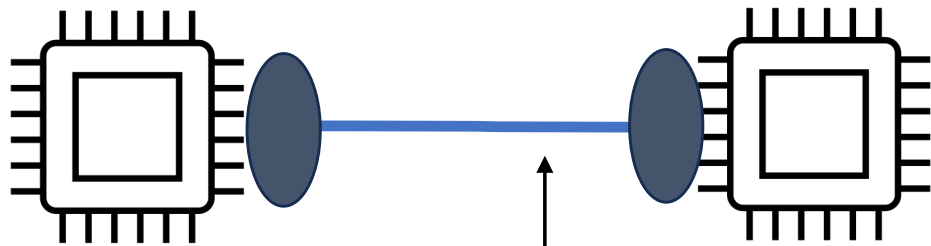
Quantum Interconnect

Code switching, Inter-code logical gates

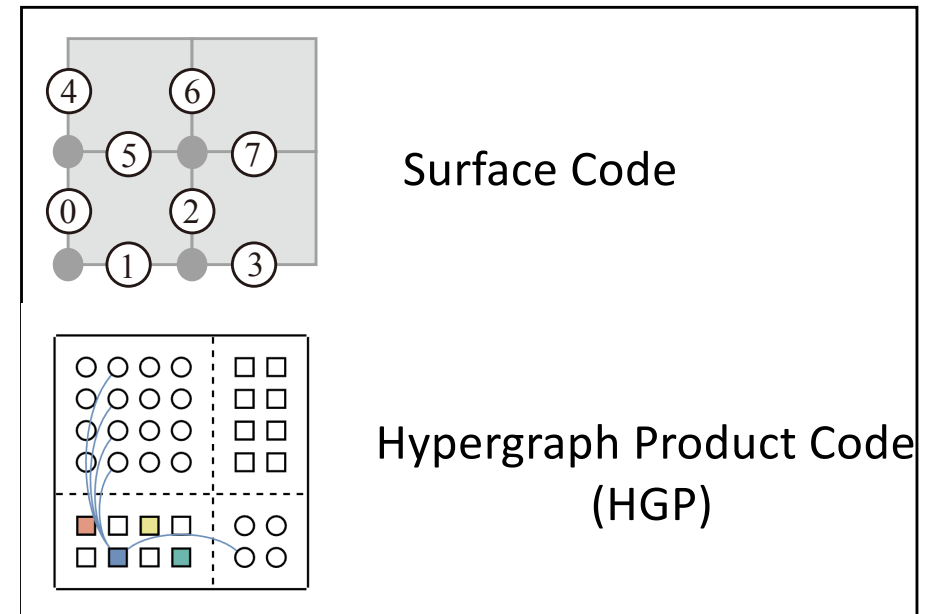
	Quantum Chip	Quantum Interconnect
Case 1	"Computation "Codes (e.g. Surface and HGP)	"Communication" codes (e.g. QRS)
Case 2	"Computation "Codes	

Hybrid codes system may introduce overhead...

Quantum Processor



Quantum Interconnect



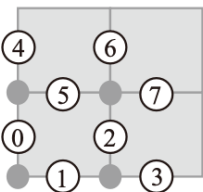
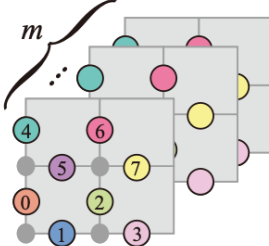
	Quantum Chip	Quantum Interconnect
Case 1	"Computation" Codes (e.g. Surface and HGP)	"Communication" codes (e.g. QRS)
Case 2	"Computation" Codes (e.g. Surface and HGP)	

Scenario (A) Multiple Code Words

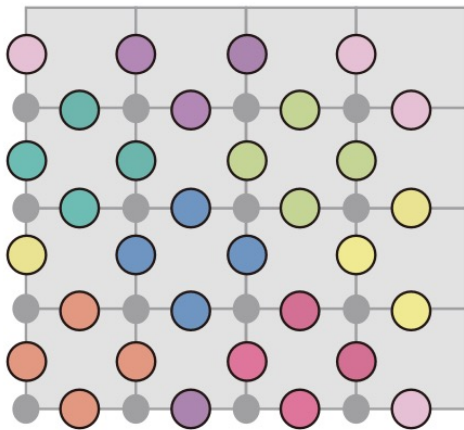
2

Sending m code words

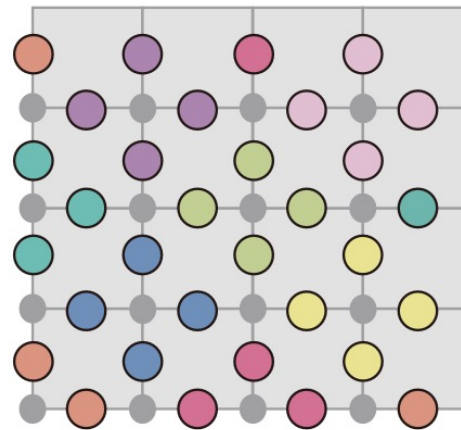
- m times better throughput
- No classical correlation inside a code

Scenarios	 without multiplexing	 (A)
Code parameters	$[[2d^2, 1, d]]$	$[[2d^2, 1, d]]$
Number of Codes	1	m
Number of Data Qubits	$2d^2$	$2md^2$
Number of Photons	$2d^2$	$2d^2$
Logical Error Rate	-	Same as without quantum multiplexing

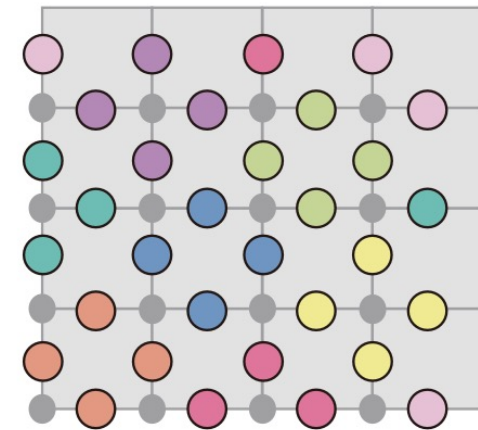
(v) Stabilizer based



Z-stabilizer



X-stabilizer

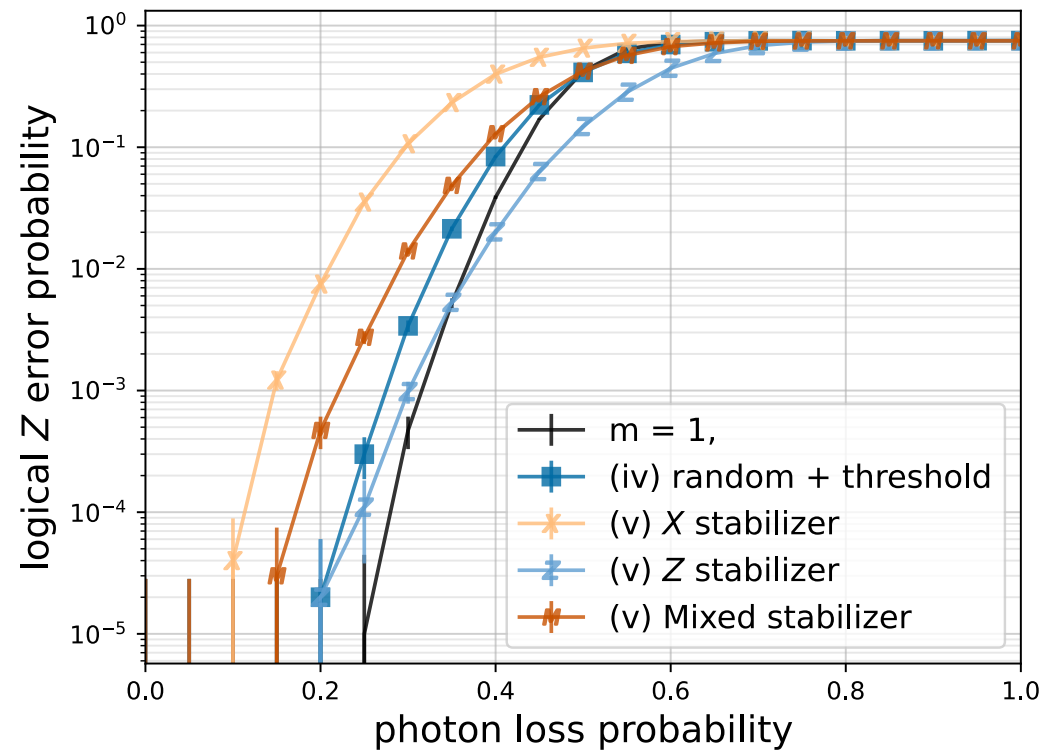


Mixed

Assignment strategies for QM

2

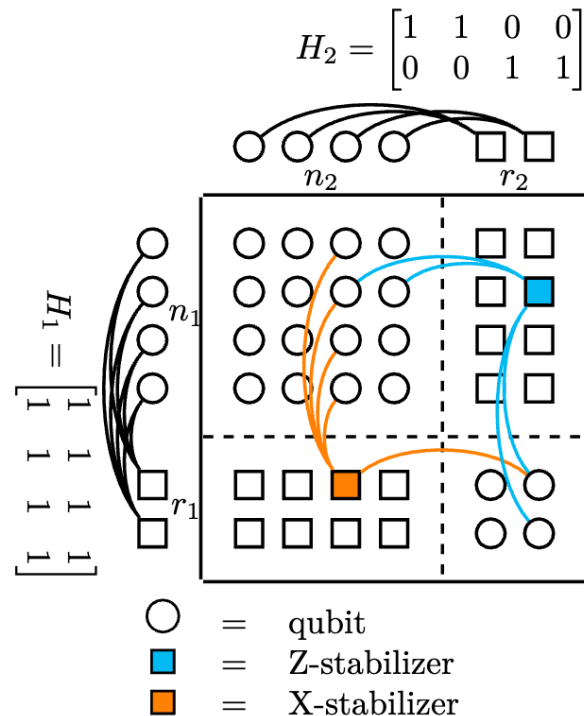
$$m = 4$$



- May be useful when using biased code or error is biased

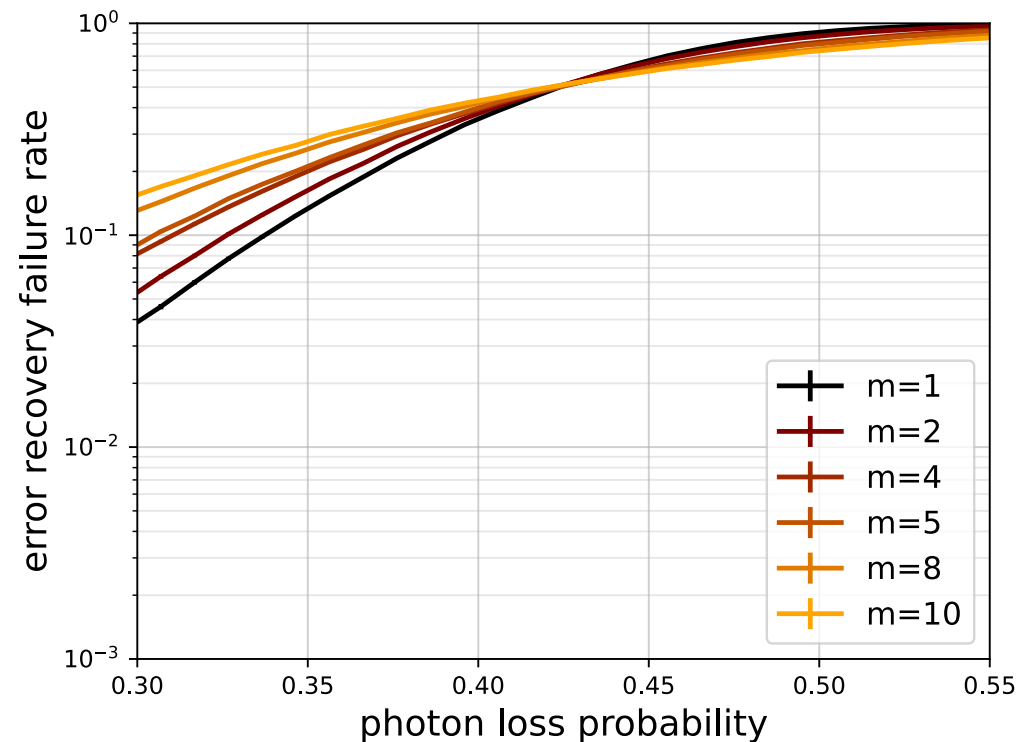
Hypergraph Product (HGP) Codes

- Asymptotically finite rate
- Can be regarded as Generalized surface code
Product of two ring graphs



$$H_X = \begin{bmatrix} 1000100010001000 & 1000 \\ 0100010001000100 & 1000 \\ 0010001000100010 & 0100 \\ 0001000100010001 & 0100 \\ 1000100010001000 & 0010 \\ 0100010001000100 & 0010 \\ 0010001000100010 & 0001 \\ 0001000100010001 & 0001 \end{bmatrix} \quad H_Z = \begin{bmatrix} 1100000000000000 & 1010 \\ 0011000000000000 & 0101 \\ 0000110000000000 & 1010 \\ 0000001100000000 & 0101 \\ 0000000011000000 & 1010 \\ 0000000000110000 & 0101 \\ 0000000000001100 & 1010 \\ 0000000000000011 & 0101 \end{bmatrix}$$

With random assignment



Non-ML Decoder

Maximally-likelihood Decoder for HGP (Gaussian elimination)

$$O(n^3)$$

Combined decoder (peeling + pruned peeling + VH)[23]

$$O(n^2) \text{ or } O(n^{1.5}) \text{ for probabilistic version}$$

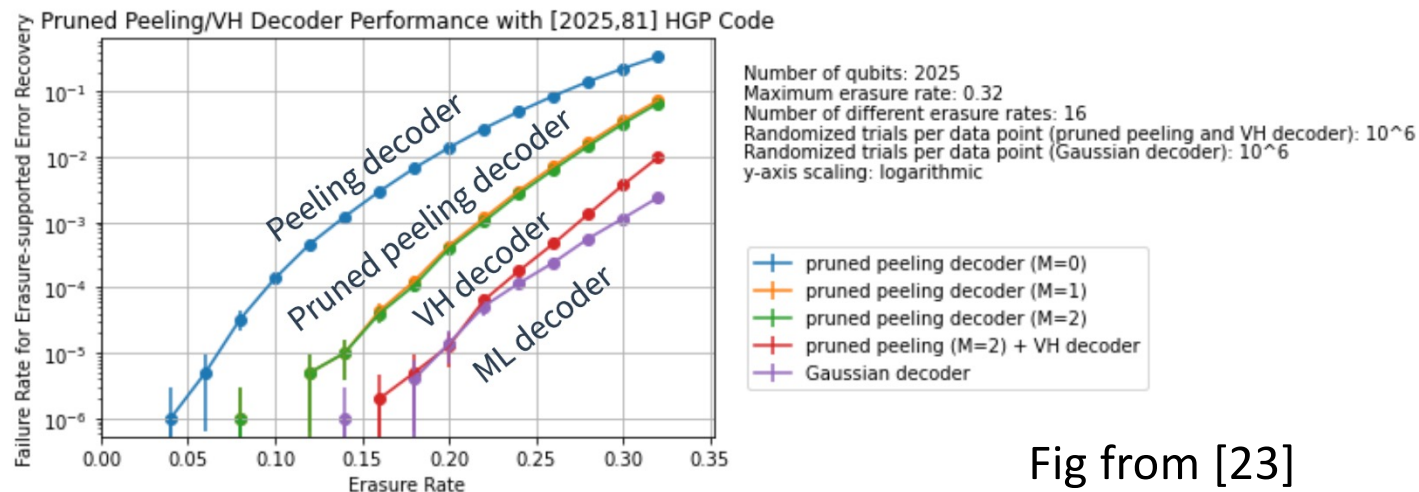


Fig from [23]

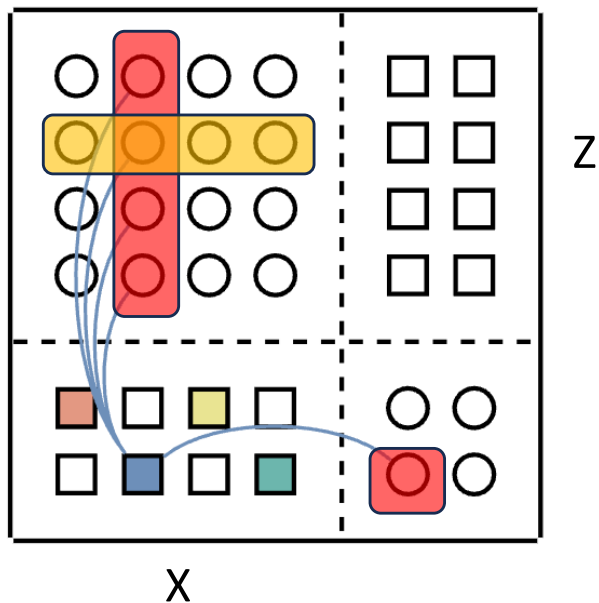
[23]N. Connolly, V. Londe, A. Leverrier, and N. Delfosse, “Fast erasure decoder for hypergraph product codes,” Quantum, vol. 8, p. 1450, Aug. 2024, doi: 10.22331/q-2024-08-27-1450.

Decoding stopping set

Combined decoder[25] runs in $O(n^2)$

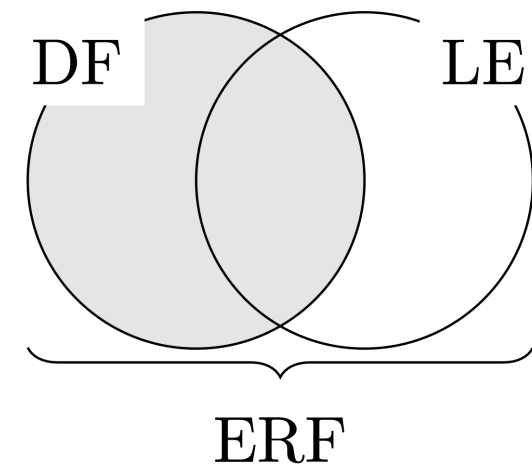
Two types of stopping set 

- Stabilizer stopping set
All qubits in support of the x/z stabilizer
- Classical stopping set 
entire row or column: classical codes

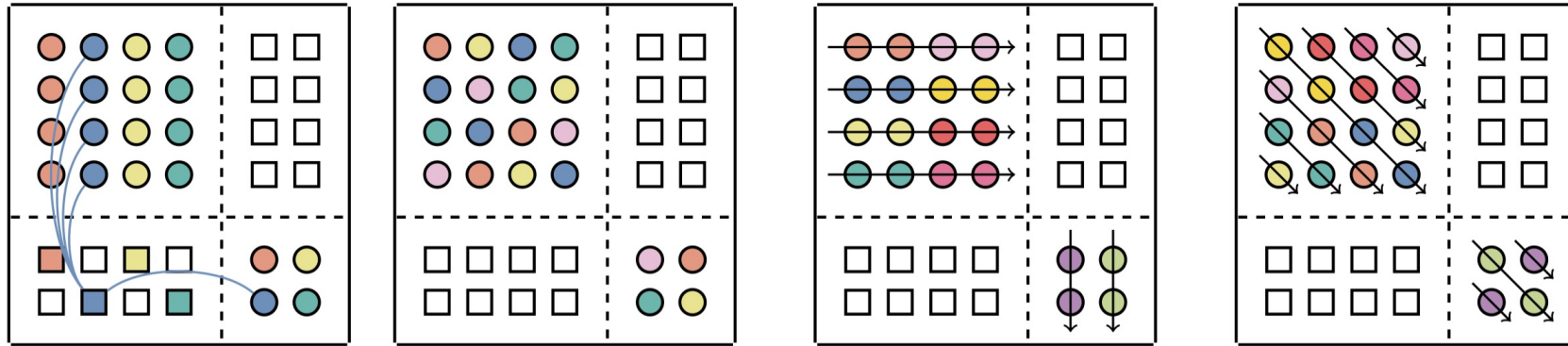


Error Recovery Failure (ERF)

- Decoding Failure (DF)
- Logical Error (LE)



Hypergraph Product (HGP) Codes

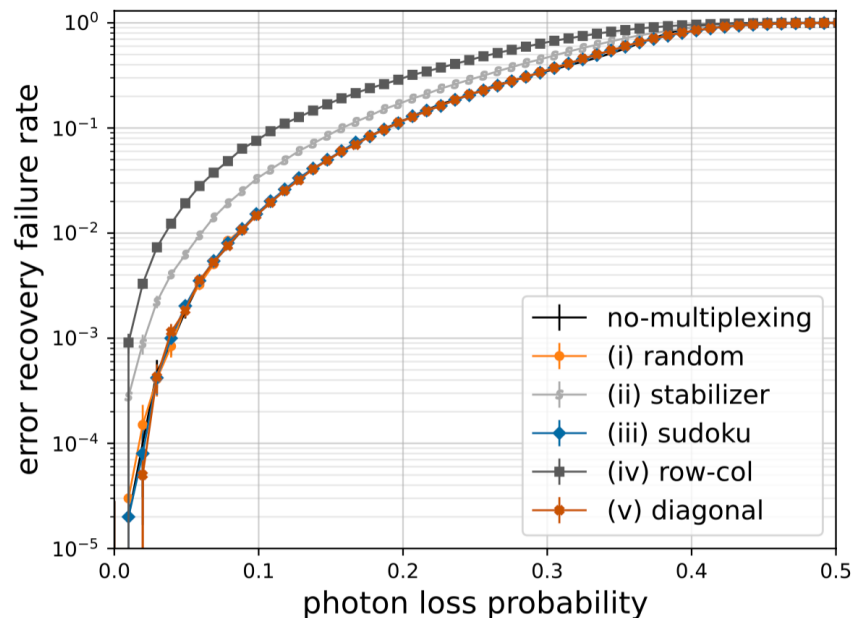


Strategy ii. Stabilizer Strategy iii. Sudoku Strategy iv. Row-Column Strategy v. Diagonal

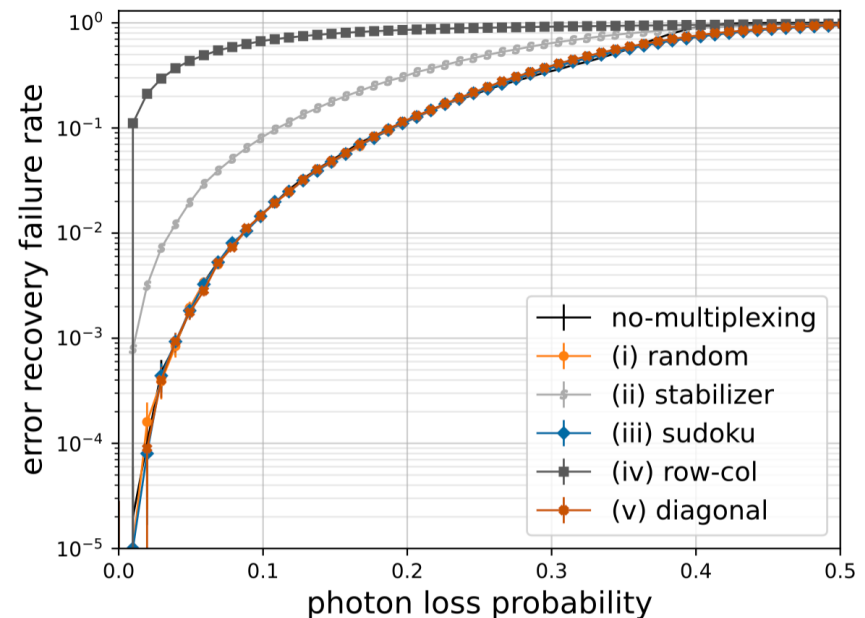
- Calculate distance is not as easy as surface codes
- Decoder-aware assignment strategies
- Without proper placement, the effect of degradation increases a lot with m

Hypergraph Product (HGP) Codes

[[512, 8]] HGP codes ($16 \times 16, 16 \times 16$)



$m = 4$



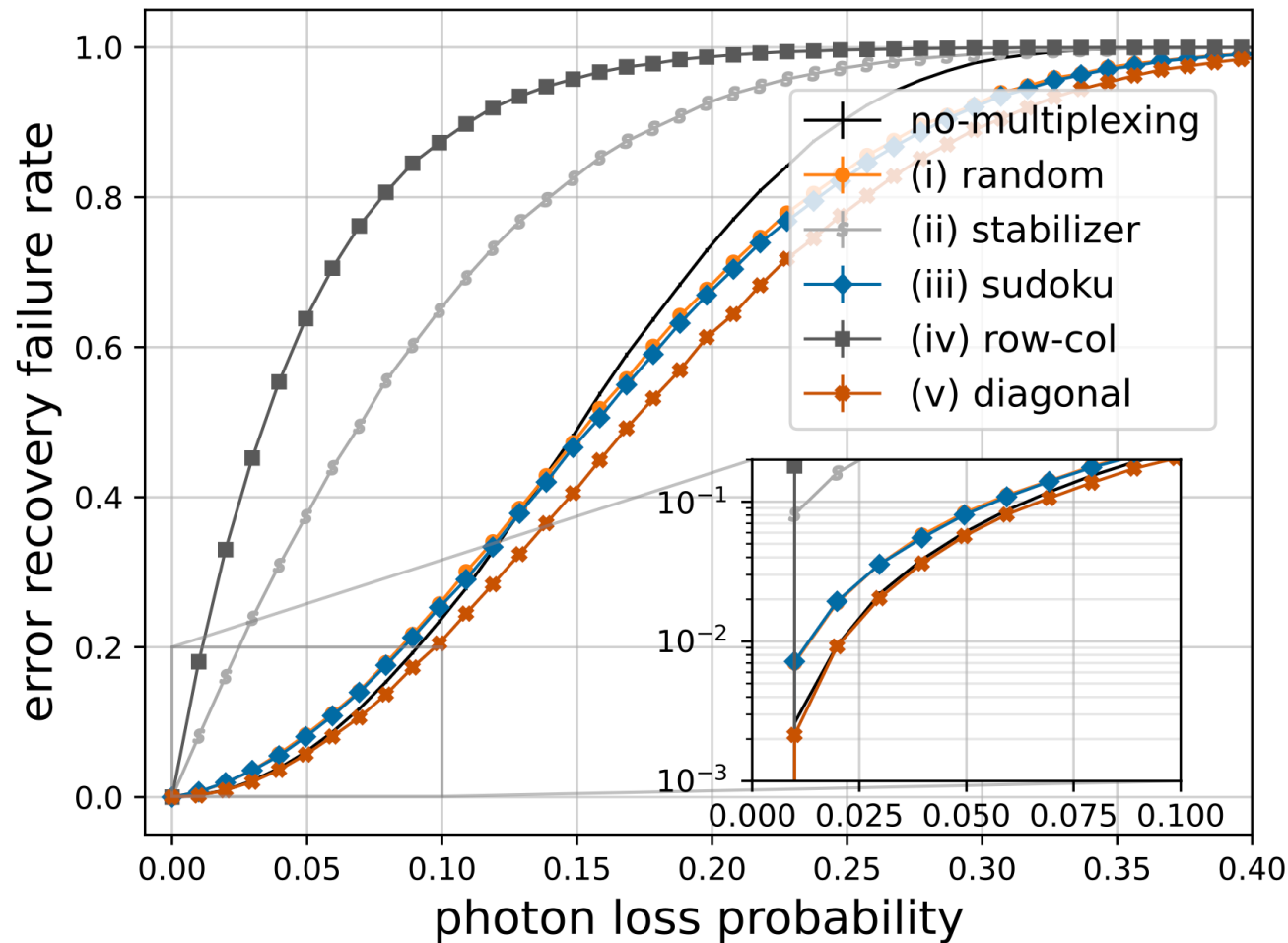
$m = 16$

(v) can achieve the same performance as without multiplexing while significantly reducing the resources required

Hypergraph Product (HGP) Codes

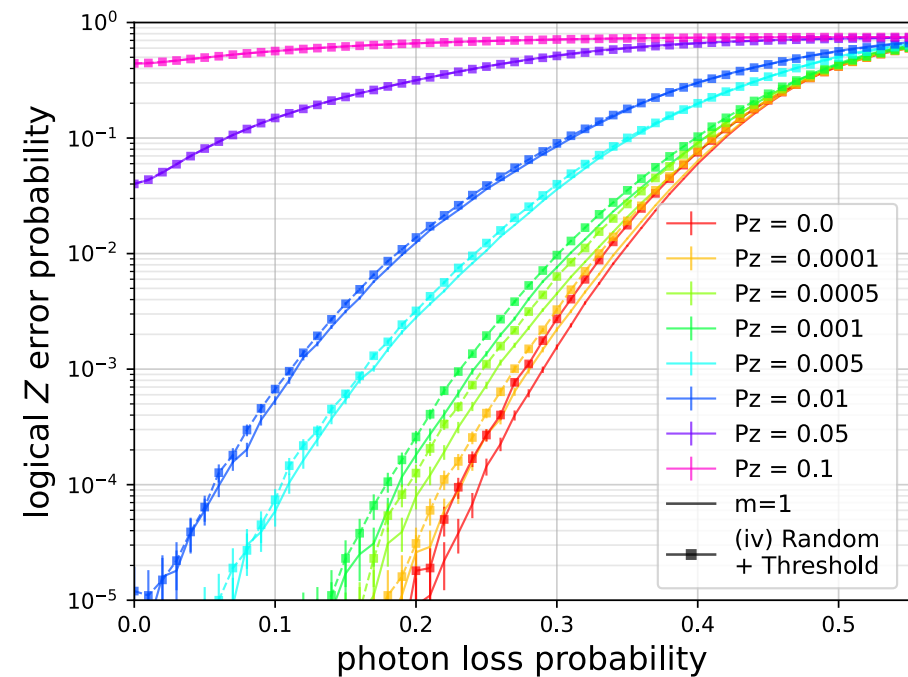
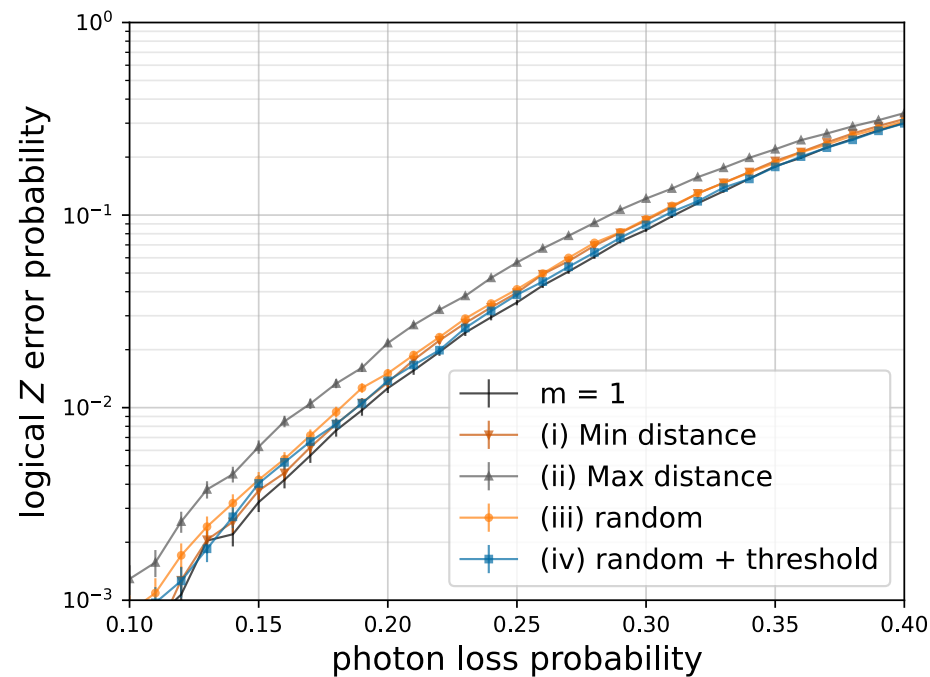
[[320, 82]] HGP codes ($16 \times 16, 8 \times 8$)

$m = 8$



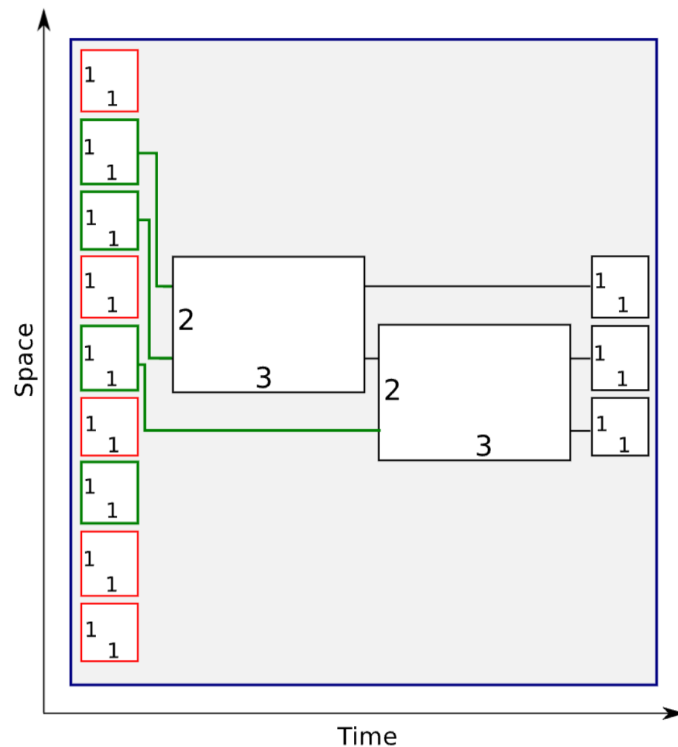
(v) is even better than no-multiplexing!

1% 1%
Combined error model (random Pauli + Loss + random Pauli)



Related Work 2: Online Scheduler for Magic

Online scheduling for routing magic states to ICM circuits



ASAP Scheduling

Red: Failure

Green: Successful Magic States

Space cost
(# of qubits)

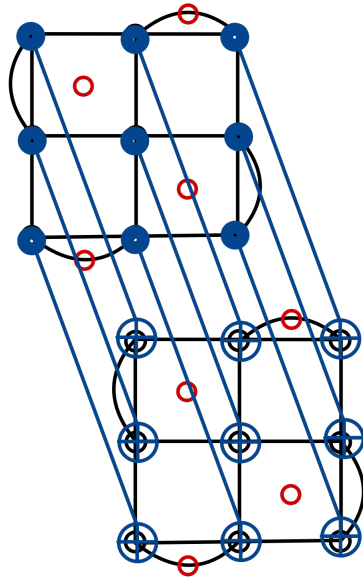
Gates

Time cost
(depth)

[21] Alexandru Paler, Austin G. Fowler, and Robert Wille. Online scheduled execution of quantum circuits protected by surface codes. *Quantum Info. Comput.*, 17(15–16):1335–1348, December 2017.

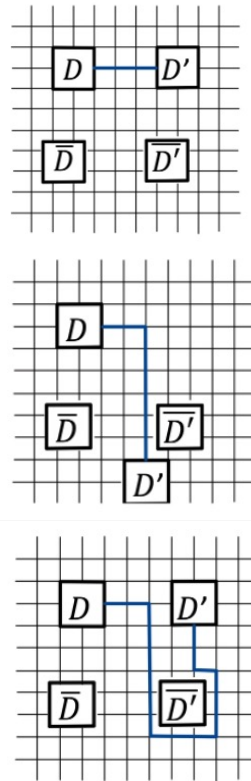
Two-qubit gates for the surface codes

Transversal gate



arbitrary CSS has transversal implementation

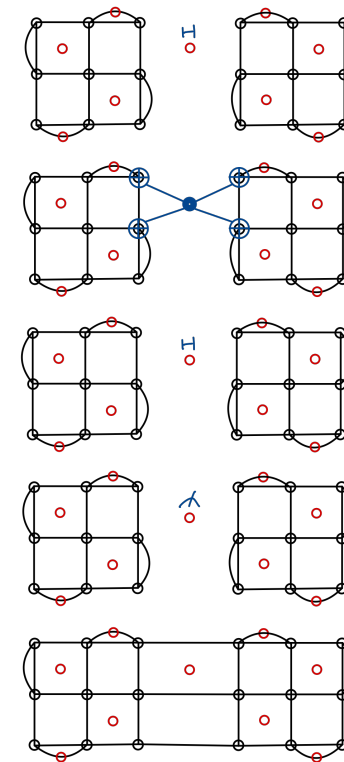
Defect braiding[18]



Larger decoding region
2D implementation

Lattice surgery[19]

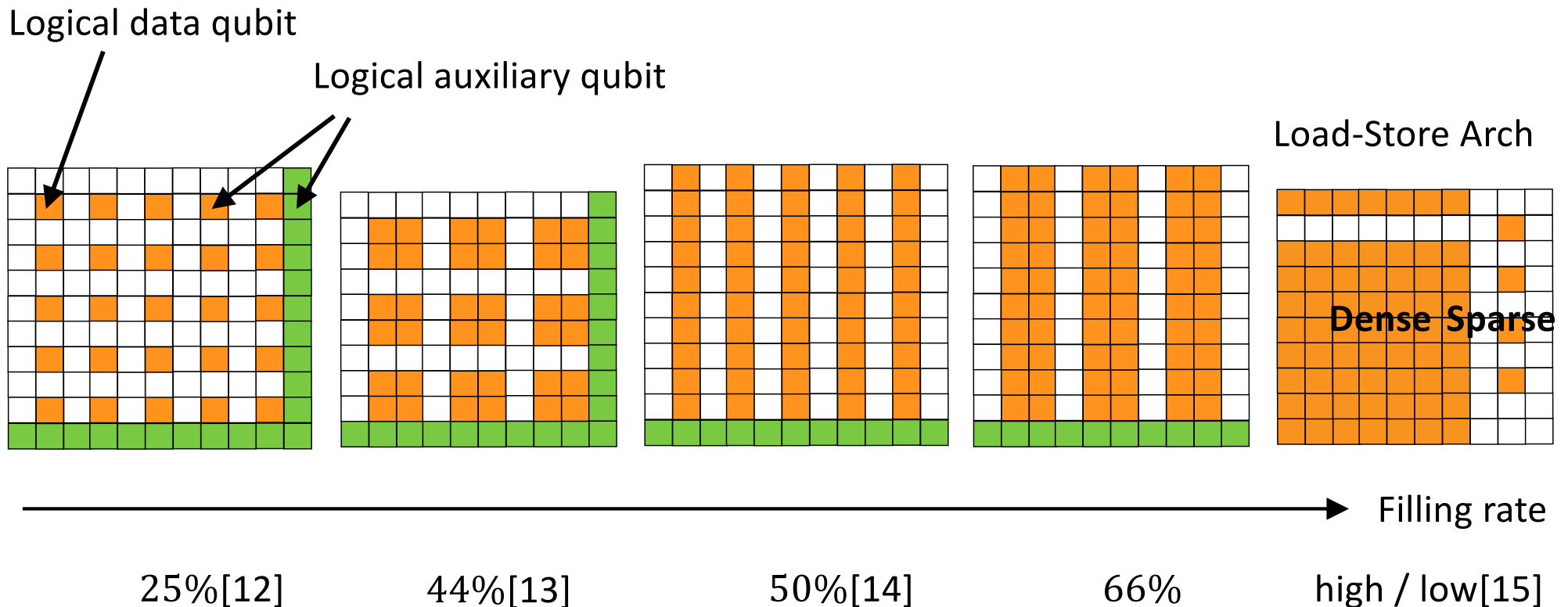
Merge • split



2D implementation

- [18] Fowler, Austin G., et al. "Surface codes: Towards practical large-scale quantum computation." *Physical Review A—Atomic, Molecular, and Optical Physics* 86.3 (2012): 032324.
 [19] Horsman, Dominic, et al. "Surface code quantum computing by lattice surgery." *New Journal of Physics* 14.12 (2012): 123011.

Floorplan of lattice surgery based FTQC



Sparse floorplan allows parallel gates

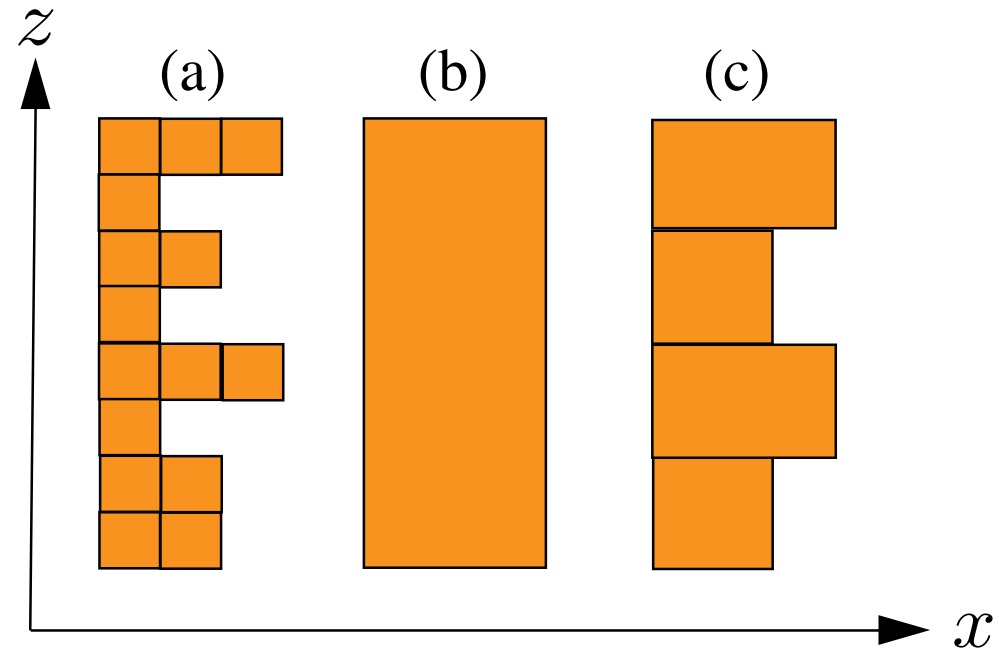
[12]Beverland, Michael, Vadym Kliuchnikov, and Eddie Schoute. "Surface code compilation via edge-disjoint paths." *PRX Quantum* 3.2 (2022): 020342.

[13]Chamberland, Christopher, and Earl T. Campbell. "Universal quantum computing with twist-free and temporally encoded lattice surgery." *PRX Quantum* 3.1 (2022): 010331.

[14]Lee, Joonho, et al. "Even more efficient quantum computations of chemistry through tensor hypercontraction." *PRX Quantum* 2.3 (2021): 030305.

[15]Kobori, Takumi, et al. "LSQCA: Resource-Efficient Load/Store Architecture for Limited-Scale Fault-Tolerant Quantum Computing." *arXiv preprint arXiv:2412.20486* (2024).

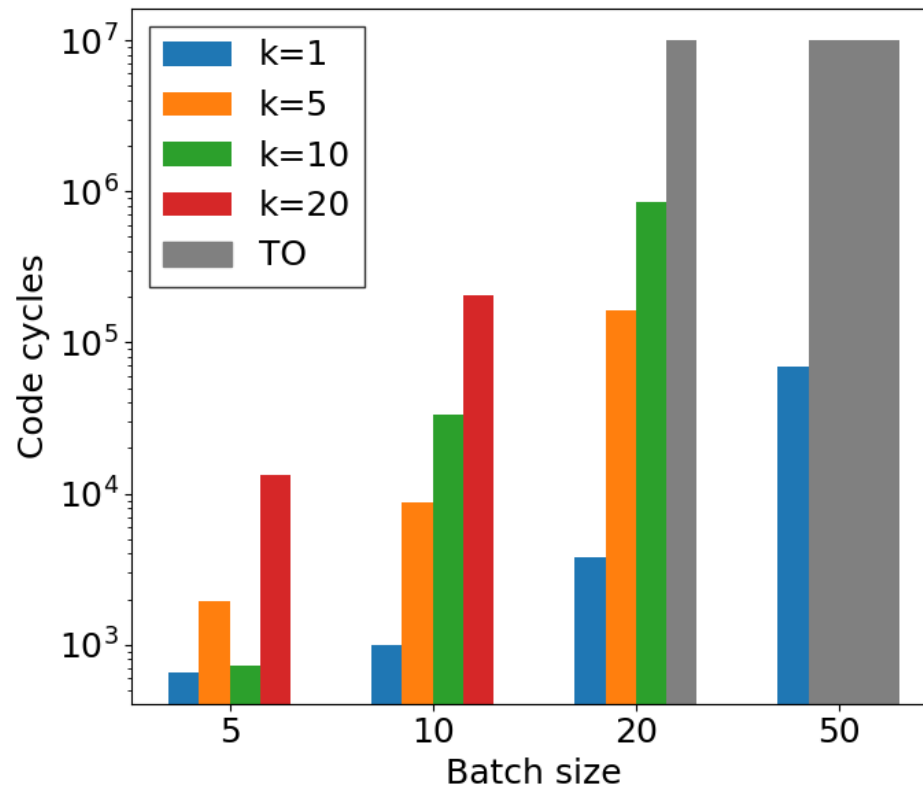
Preprocessing



- (a) Original program
- (b) Cuboid
- (c) k -cuboid

Responsiveness of ILP solver

- k : fineness of division
- Batch size: number of job requests given at once



100 μ s / code cycle
20 $\times$ 20, QASM Bench Medium

- Roughly response time increases in proportion to batch size & k , but computation time is difficult to predict!

Scheduler Responsiveness

Code cycle to the end of all program calculations

Chip size	greedy-poly	ILP-poly	corner-greedy	ILP-1-cuboid	ILP-5-cuboid	Naive
20 × 20	TO	TO	20337	35379	77891	74640

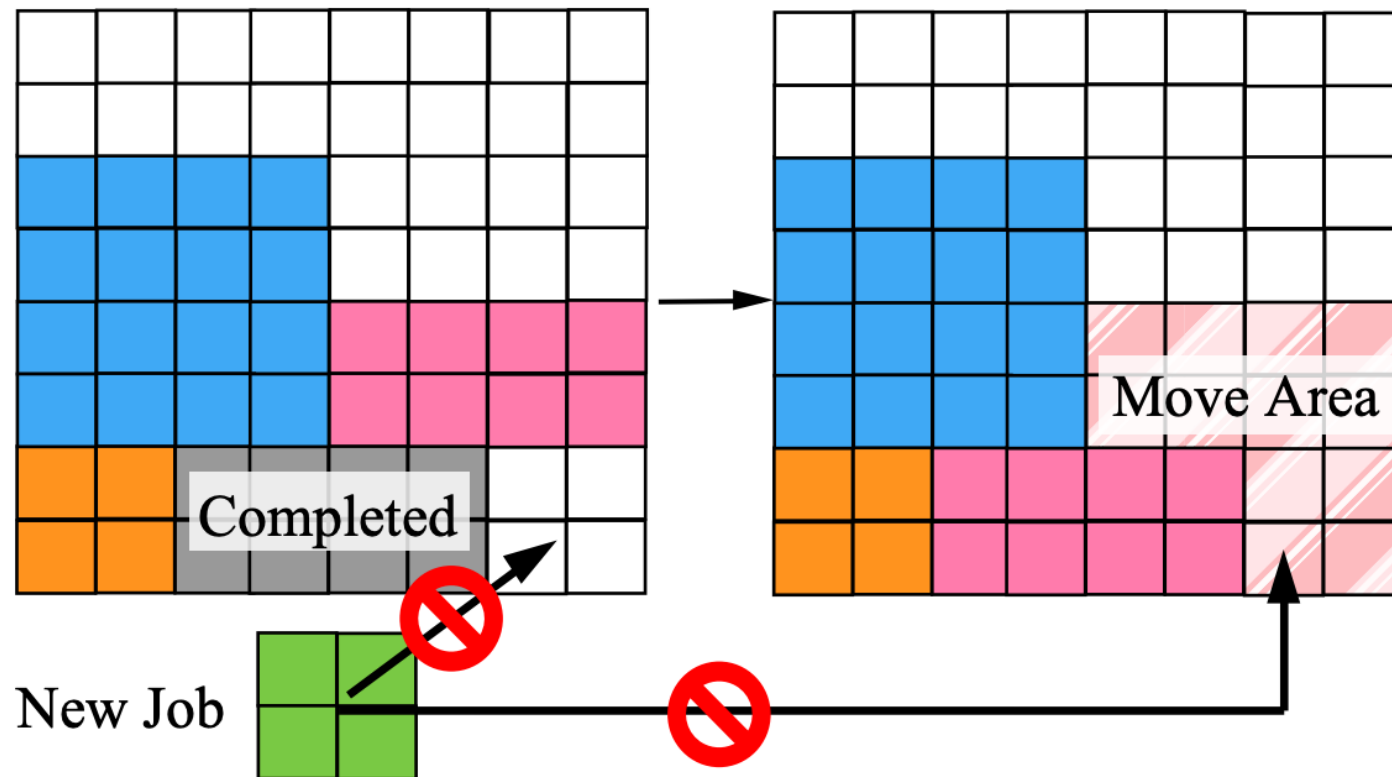
TO: Timeout (schedule not completed within 2 minutes)

20×20 Medium-size QASM Bench

	Naive Greedy	Corner Greedy	ILP
Responsiveness(μs)	84141500	980	224520

Defragmentation

Job movement paths are occupied during defragmentation



Parameter	Value
Code cycle	1 μ s
Code distance d	31
Chip size $W \times H$	20×20
Batch size B	5
Defrag interval I	$2 \times 10^4 \times d \mu$ s

Random circuit to create data set

Type	w, h	l	Constraint
I	[5, 10]	$[1 \times 10^4, 2 \times 10^4]$	
II	[5, 10]	$[4 \times 10^4, 6 \times 10^4]$	
III	[5, 10]	$[8 \times 10^4, 1 \times 10^5]$	
IV	[10, 20]	$[1 \times 10^4, 2 \times 10^4]$	$101 \leq w \times h \leq 200$
V	[10, 20]	$[4 \times 10^4, 6 \times 10^4]$	$101 \leq w \times h \leq 200$
VI	[10, 20]	$[8 \times 10^4, 1 \times 10^5]$	$101 \leq w \times h \leq 200$

small

Job size

big

Class	#Request	Ratio of each data type
A		I: 50%, others : 10% each
B		II: 50%, others : 10% each
C		III: 50%, others : 10% each
D		IV: 50%, others : 10% each
E	300	V: 50%, others : 10% each
F		VI: 50%, others : 10% each
G		All: 16.6% each (uniform ratio)
H		I, II, III: 30% each, IV,V,VI: 3.33% each
I		I, II, III: 3.33% each, IV,V,VI: 30% each